

```
In [1]: import seaborn as sns
import matplotlib.pyplot as plt
import scipy.stats as st
%matplotlib inline

sns.set(style="whitegrid")
```

```
In [2]: import warnings
warnings.filterwarnings('ignore')
```

```
In [3]: import pandas as pd
import numpy as np
```

```
In [4]: df = pd.read_csv(r"C:\Users\LIKHITH\OneDrive\Documents\heart.csv")
```

```
In [5]: df
```

```
Out[5]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2
...	...	...	...	...	...	...	...	...	...	...	...	...	...
298	57	0	0	140	241	0	1	123	1	0.2	1	0	3
299	45	1	3	110	264	0	1	132	0	1.2	1	0	3
300	68	1	0	144	193	1	1	141	0	3.4	1	2	3
301	57	1	0	130	131	0	1	115	1	1.2	1	1	3
302	57	0	1	130	236	0	0	174	0	0.0	1	1	2

303 rows × 14 columns



import libraries

```
In [6]: import os
for dirname, _, filenames in os.walk('C:/Users/LIKHITH/OneDrive/Documents'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# Any results you write to the current directory are saved as output.
```

```

C:/Users/LIKHITH/OneDrive/Documents/app.py
C:/Users/LIKHITH/OneDrive/Documents/code.py
C:/Users/LIKHITH/OneDrive/Documents/dataset_1.csv
C:/Users/LIKHITH/OneDrive/Documents/heart.csv
C:/Users/LIKHITH/OneDrive/Documents/import streamlit as st.py
C:/Users/LIKHITH/OneDrive/Documents/intro.py
C:/Users/LIKHITH/OneDrive/Documents/Movie-Rating.csv
C:/Users/LIKHITH/OneDrive/Documents/movie.csv
C:/Users/LIKHITH/OneDrive/Documents/radhakrishna.jpg
C:/Users/LIKHITH/OneDrive/Documents/rating.csv
C:/Users/LIKHITH/OneDrive/Documents/Rawdata.xlsx
C:/Users/LIKHITH/OneDrive/Documents/sql_dataset.pdf
C:/Users/LIKHITH/OneDrive/Documents/sql_dataset.sql
C:/Users/LIKHITH/OneDrive/Documents/tag.csv
C:/Users/LIKHITH/OneDrive/Documents/test.py
C:/Users/LIKHITH/OneDrive/Documents/test1.py
C:/Users/LIKHITH/OneDrive/Documents/test2.py
C:/Users/LIKHITH/OneDrive/Documents/tkintertext.py
C:/Users/LIKHITH/OneDrive/Documents/WhatsApp Image 2025-11-16 at 19.36.19_04fe1a9f.jpg
C:/Users/LIKHITH/OneDrive/Documents/WhatsApp Image 2025-11-16 at 19.36.19_9aa6b3bd.jpg
C:/Users/LIKHITH/OneDrive/Documents/anaconda_projects/db/project_filebrowser.db
C:/Users/LIKHITH/OneDrive/Documents/__pycache__/tkinter.cpython-312.pyc

```

EDA

In [7]: `df.shape`

Out[7]: (303, 14)

In [8]: `df.head()`

Out[8]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	ti
0	63	1	3	145	233	1	0	150	0	2.3	0	0	1	
1	37	1	2	130	250	0	1	187	0	3.5	0	0	2	
2	41	0	1	130	204	0	0	172	0	1.4	2	0	2	
3	56	1	1	120	236	0	1	178	0	0.8	2	0	2	
4	57	0	0	120	354	0	1	163	1	0.6	2	0	2	

In [9]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 303 entries, 0 to 302
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   age         303 non-null   int64
 1   sex         303 non-null   int64
 2   cp          303 non-null   int64
 3   trestbps    303 non-null   int64
 4   chol        303 non-null   int64
 5   fbs         303 non-null   int64
 6   restecg     303 non-null   int64
 7   thalach     303 non-null   int64
 8   exang       303 non-null   int64
 9   oldpeak     303 non-null   float64
10   slope       303 non-null   int64
11   ca          303 non-null   int64
12   thal        303 non-null   int64
13   target      303 non-null   int64
dtypes: float64(1), int64(13)
memory usage: 33.3 KB
```

```
In [10]: id(df)
```

```
Out[10]: 1416127779840
```

```
In [11]: df.dtypes
```

```
Out[11]: age          int64
sex            int64
cp             int64
trestbps       int64
chol           int64
fbs            int64
restecg        int64
thalach        int64
exang          int64
oldpeak        float64
slope          int64
ca             int64
thal           int64
target         int64
dtype: object
```

```
In [12]: df.describe()
```

Out[12]:

	age	sex	cp	trestbps	chol	fbs	restecg
count	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000	303.000000
mean	54.366337	0.683168	0.966997	131.623762	246.264026	0.148515	0.528053
std	9.082101	0.466011	1.032052	17.538143	51.830751	0.356198	0.525860
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000
25%	47.500000	0.000000	0.000000	120.000000	211.000000	0.000000	0.000000
50%	55.000000	1.000000	1.000000	130.000000	240.000000	0.000000	1.000000
75%	61.000000	1.000000	2.000000	140.000000	274.500000	0.000000	1.000000
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000

In [13]: `df.columns`

Out[13]: Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'], dtype='object')

Univariate analysis

- Our feature variable of interest is `target`.
- It refers to the presence of heart disease in the patient.
- It is integer valued as it contains two integers 0 and 1 - (0 stands for absence of heart disease and 1 for presence of heart disease).
- So, in this section, I will analyze the `target` variable.

In [14]: `df['target'].nunique()`

Out[14]: 2

In [15]: `df['target'].unique()`

Out[15]: array([1, 0])

In [16]: `df['target'].value_counts()`

Out[16]: target
 1 165
 0 138
 Name: count, dtype: int64

In [17]: `df['thal'].nunique()`

Out[17]: 4

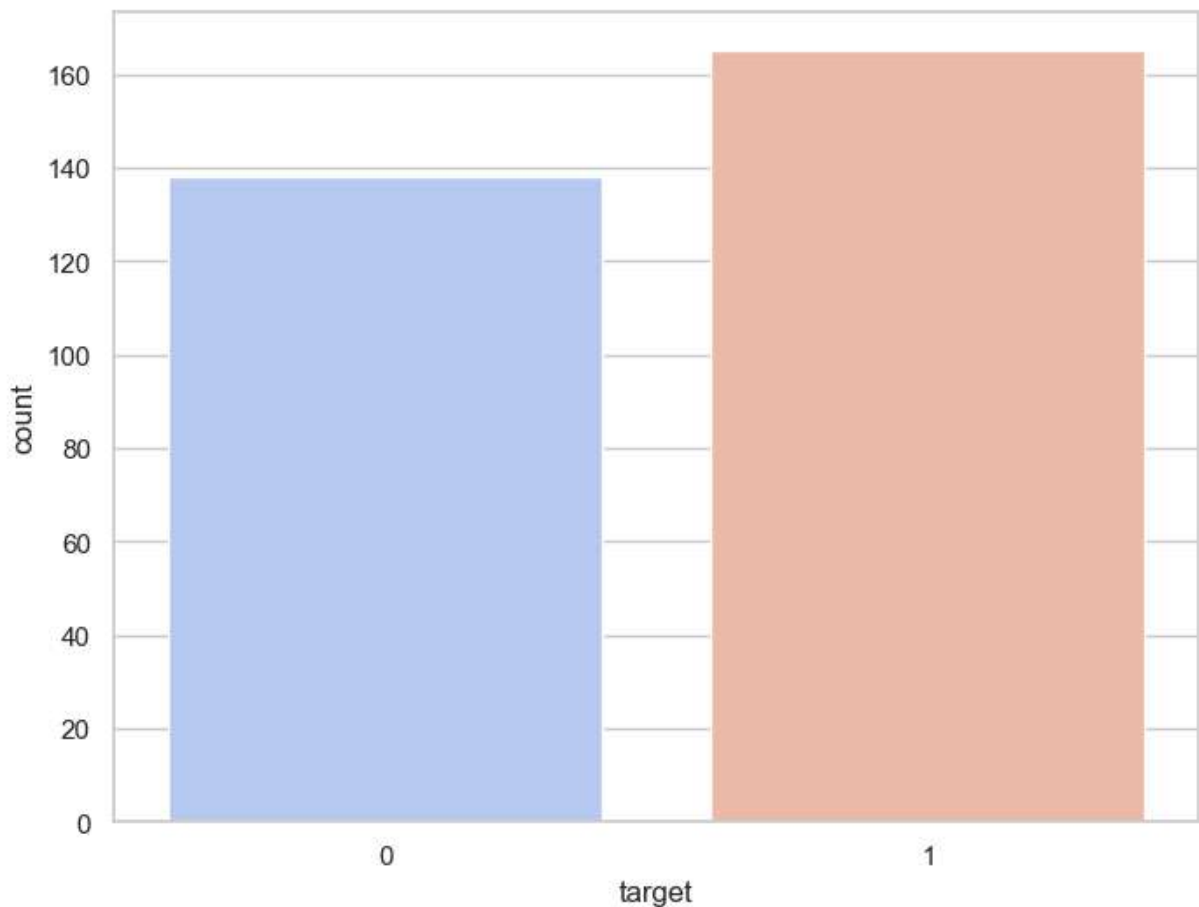
```
In [18]: df['thal'].unique()
```

Out[18]: array([1, 2, 3, 0])

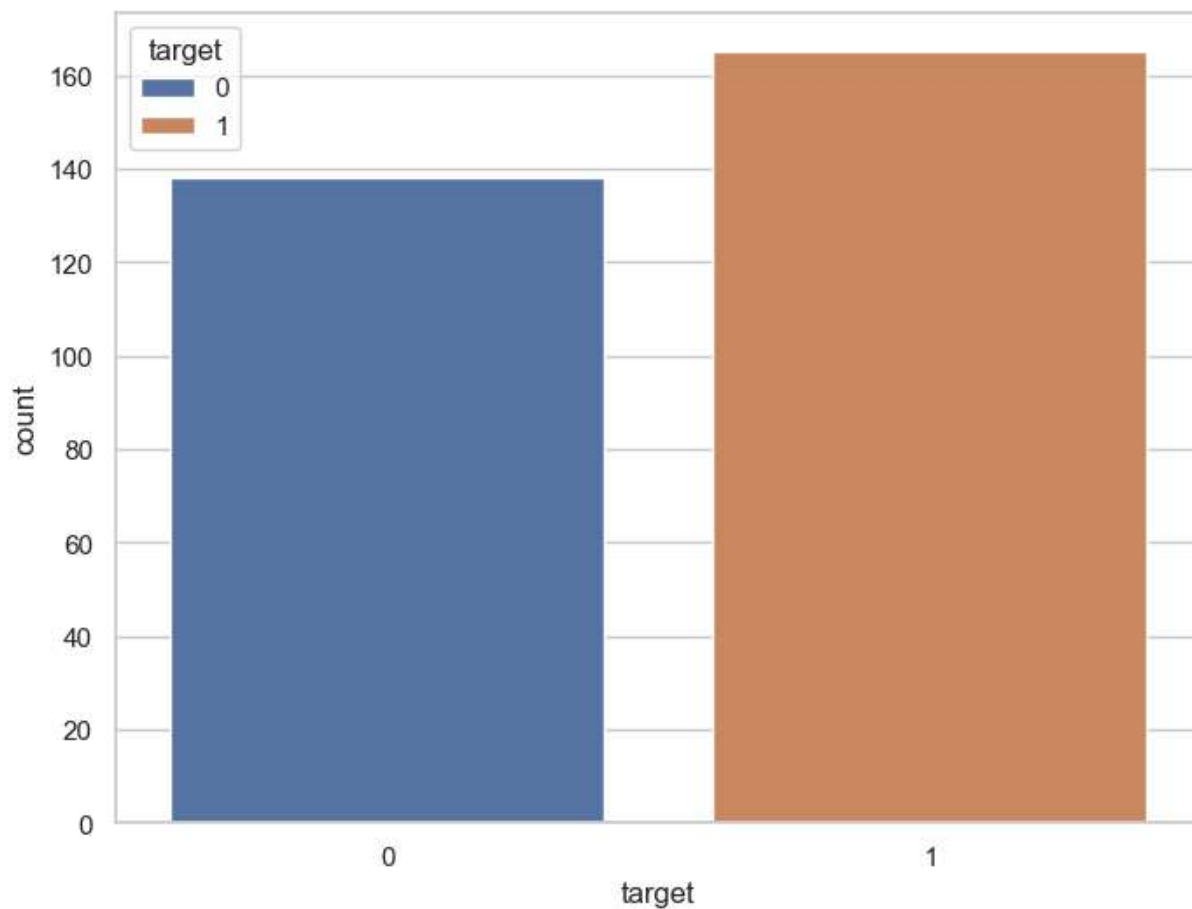
```
In [19]: df['thal'].value_counts()
```

```
Out[19]: thal
2      166
3      117
1       18
0        2
Name: count, dtype: int64
```

```
In [20]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="target", data=df,palette="coolwarm")
plt.show()
```



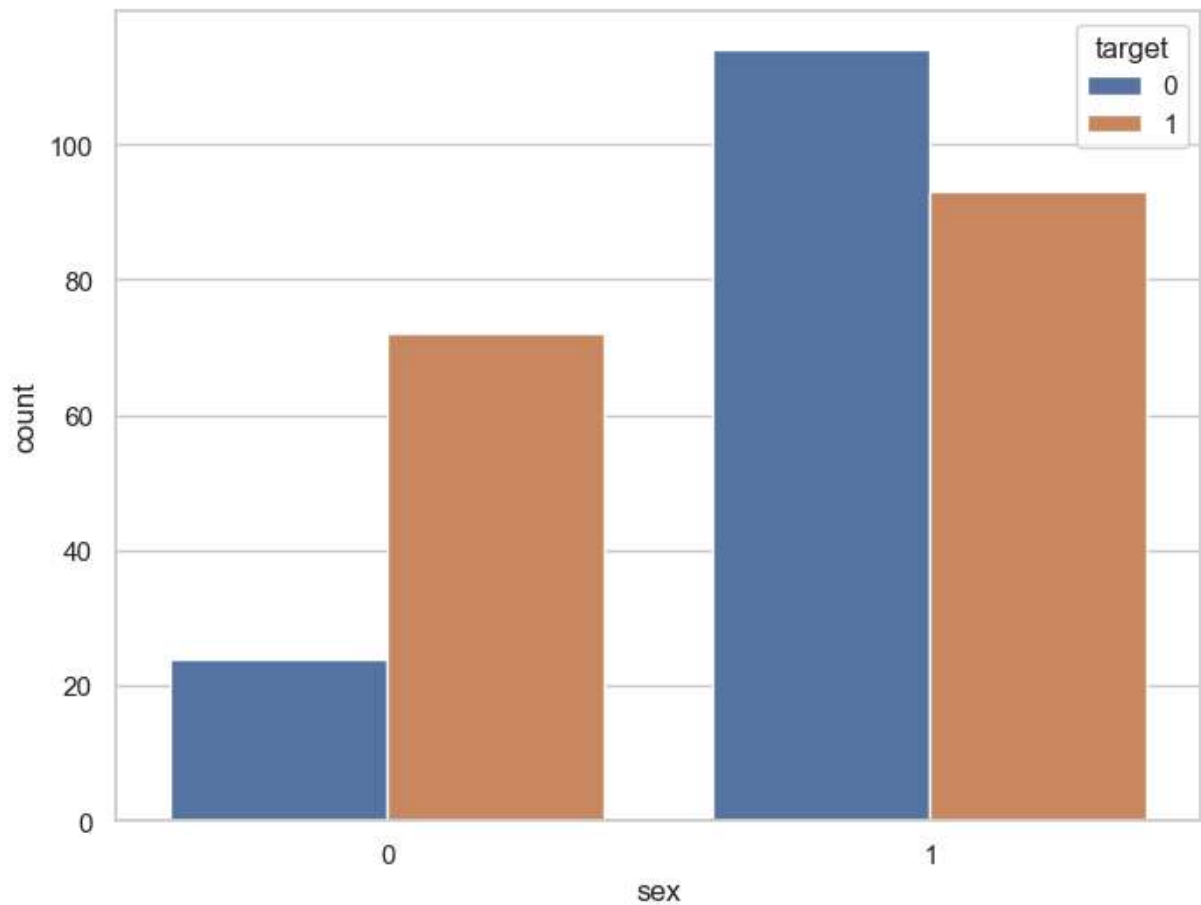
```
In [21]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="target", data=df,hue="target")
plt.show()
```



```
In [22]: df.groupby('sex')['target'].value_counts()
```

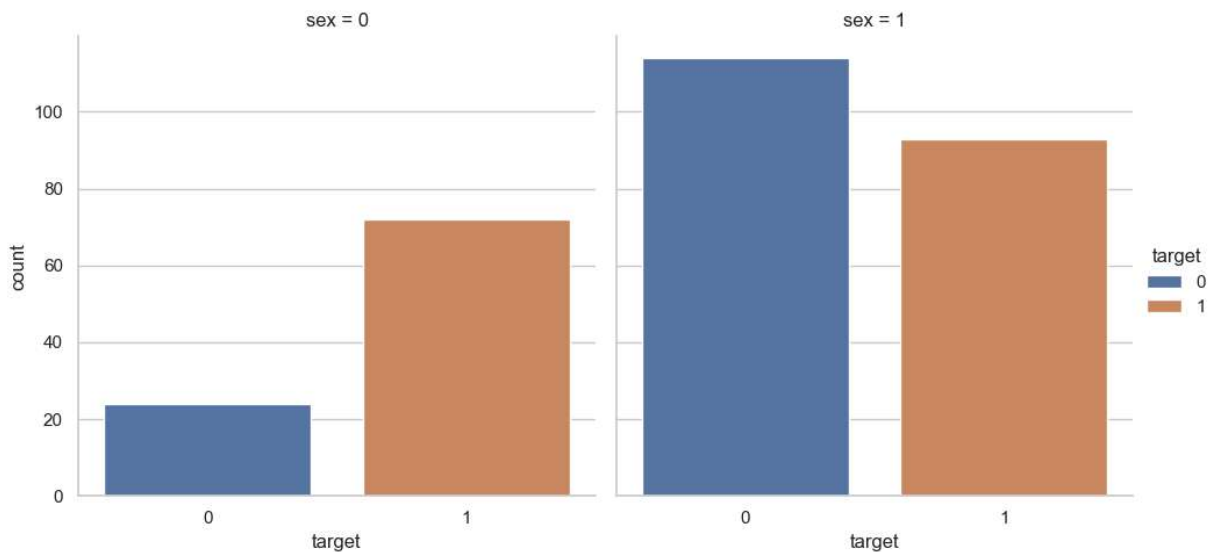
```
Out[22]: sex  target
0      1         72
      0         24
1      0        114
      1         93
Name: count, dtype: int64
```

```
In [23]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="sex", hue="target", data=df)
plt.show()
```

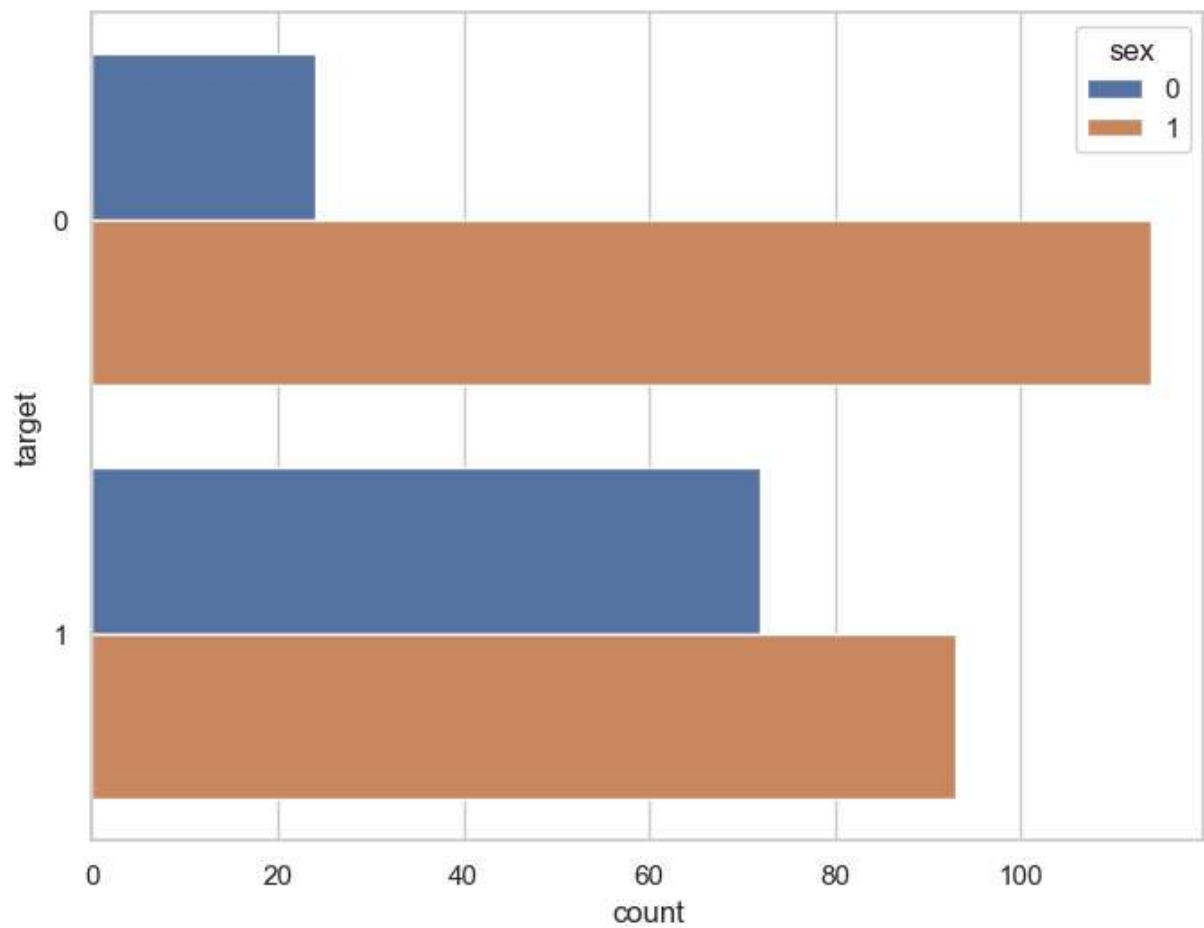


```
In [24]: ax = sns.catplot(x="target", col="sex", data=df, kind="count", height=5, aspect=1,
```

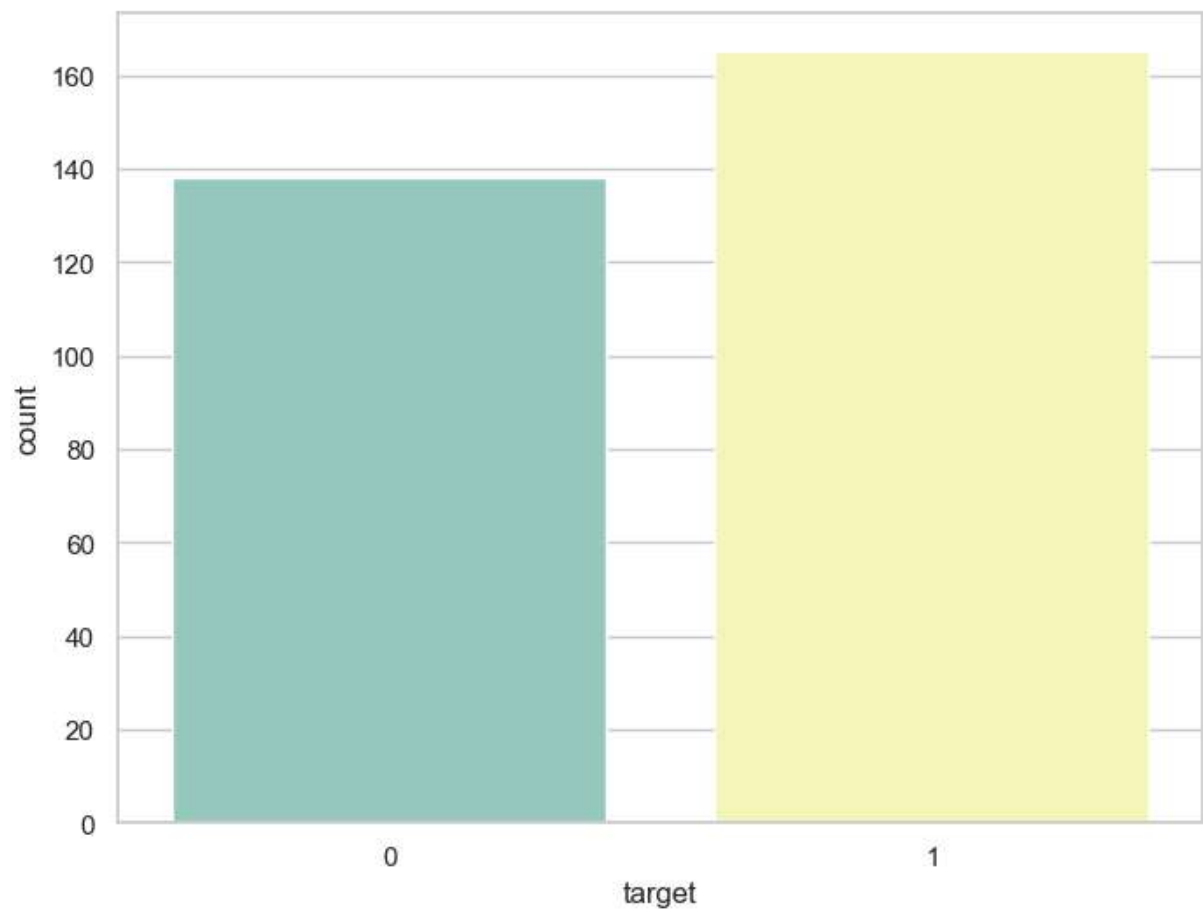
```
plt.show())
```



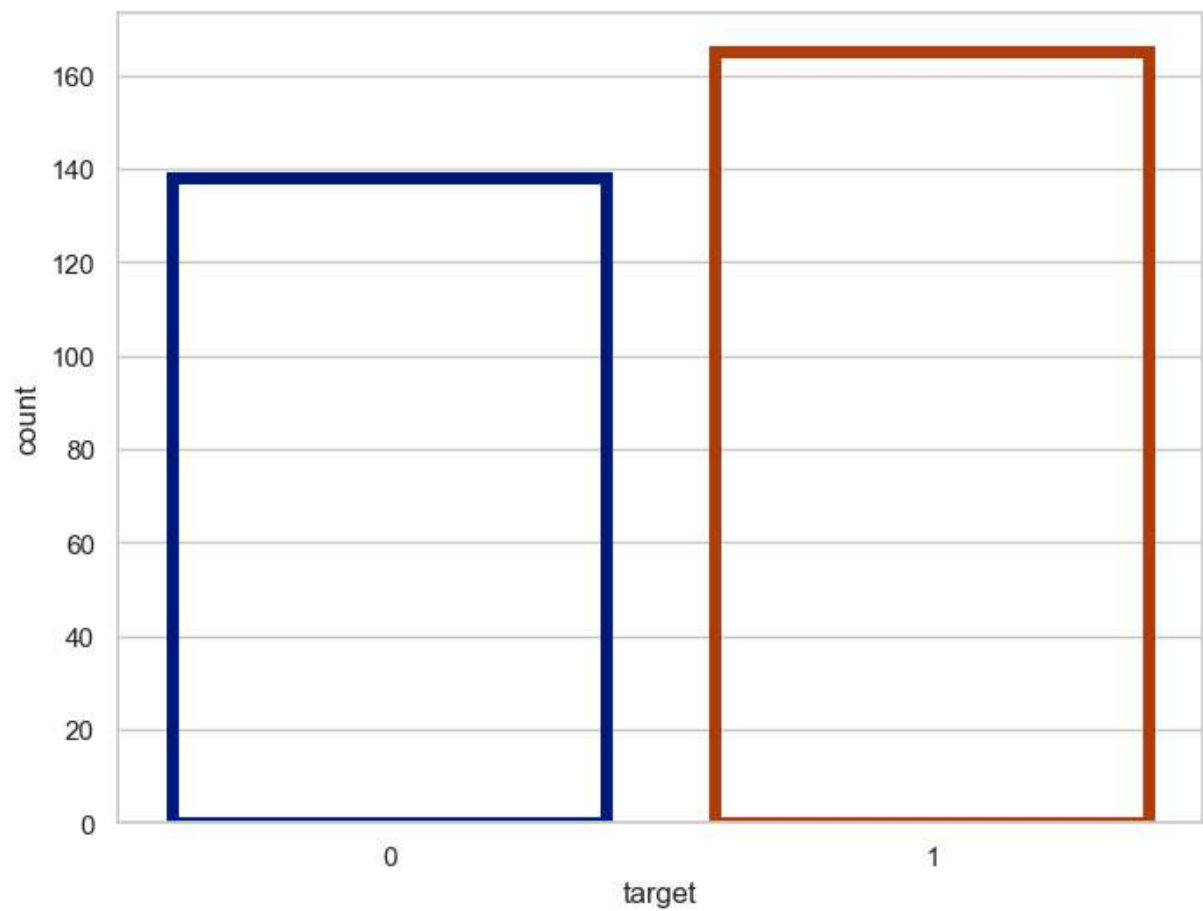
```
In [26]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(y="target", hue="sex", data=df)
plt.show()
```



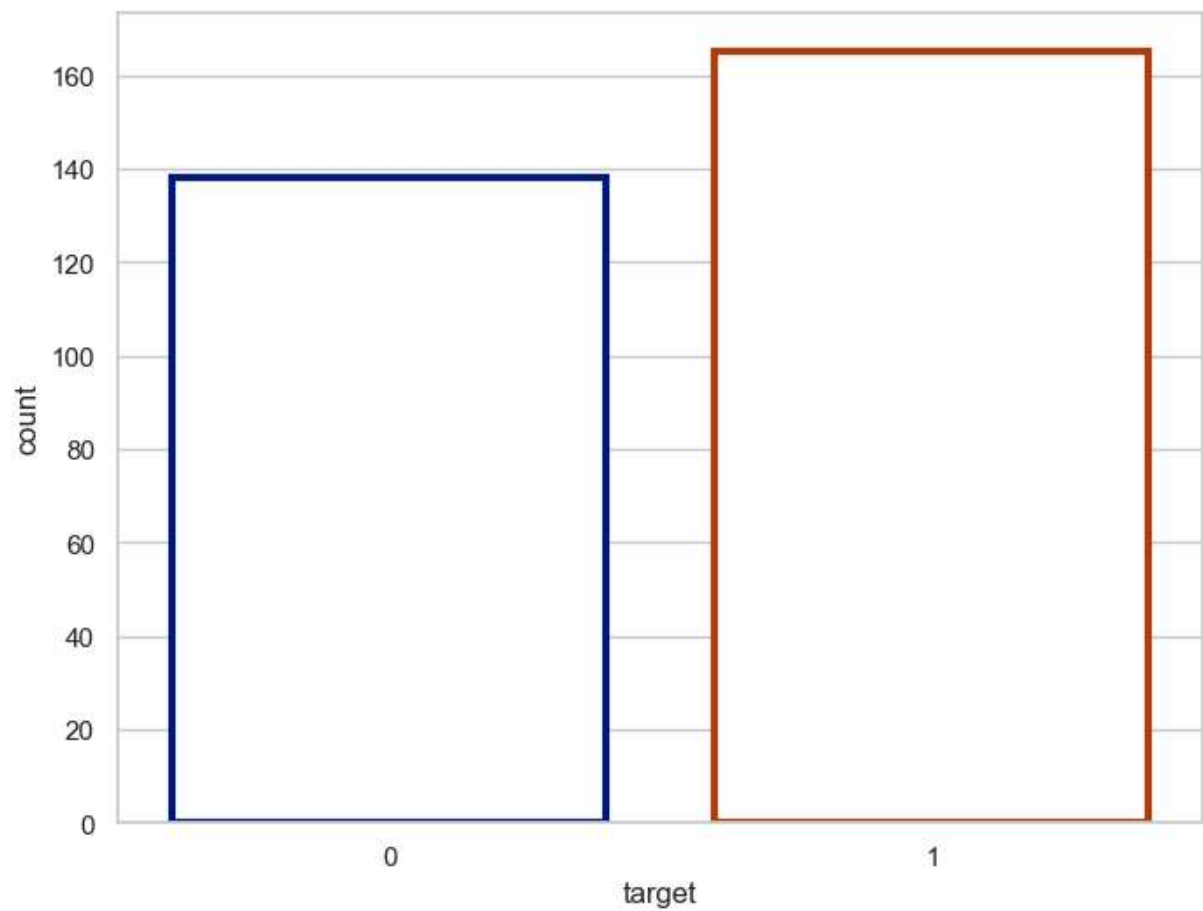
```
In [27]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="target", data=df, palette="Set3")
plt.show()
```

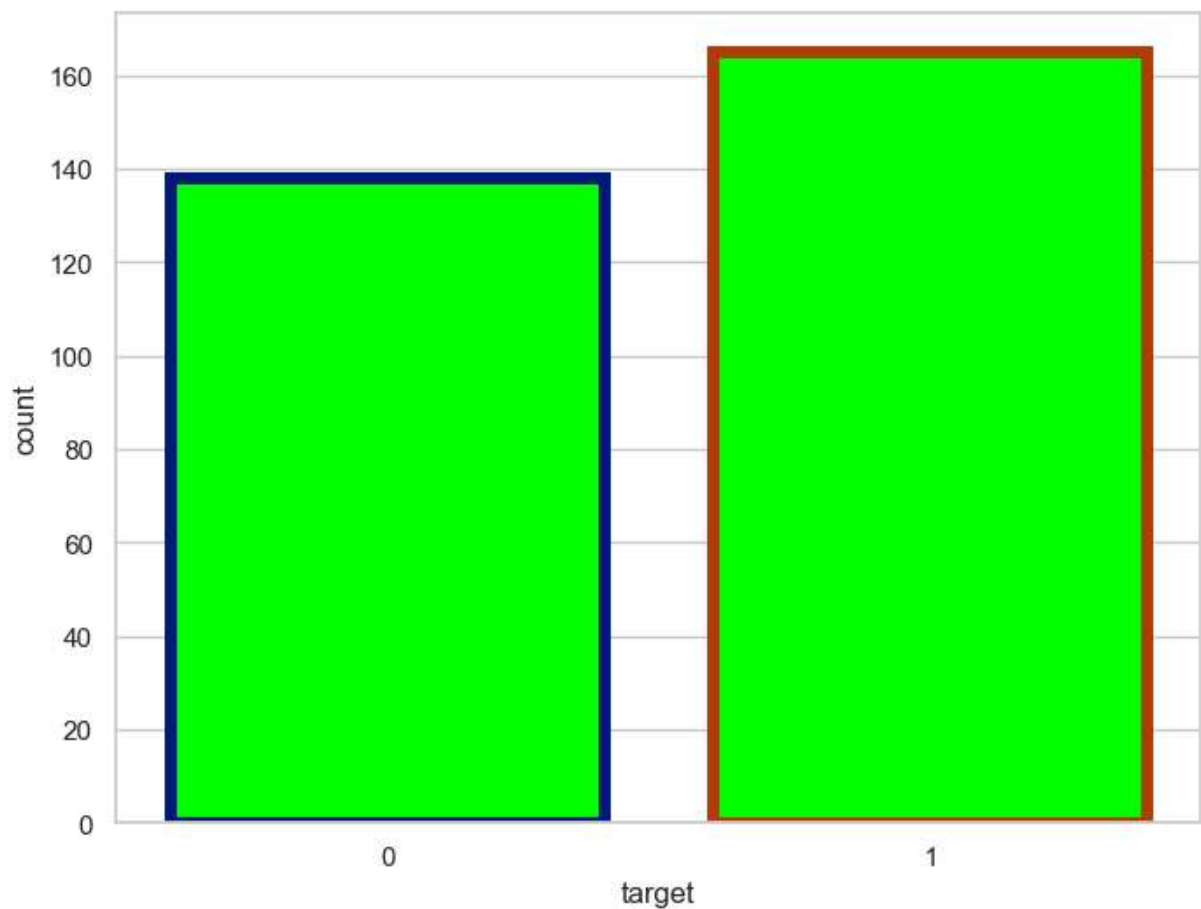
```
In [28]: f, ax = plt.subplots(figsize=(8, 6))  
ax = sns.countplot(x="target", data=df, facecolor=(0, 0, 0, 0), linewidth=5, edgeco  
plt.show()
```



```
In [29]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="target", data=df, facecolor=(1,1,1,1), linewidth=3, edgecolor=
plt.show()
```



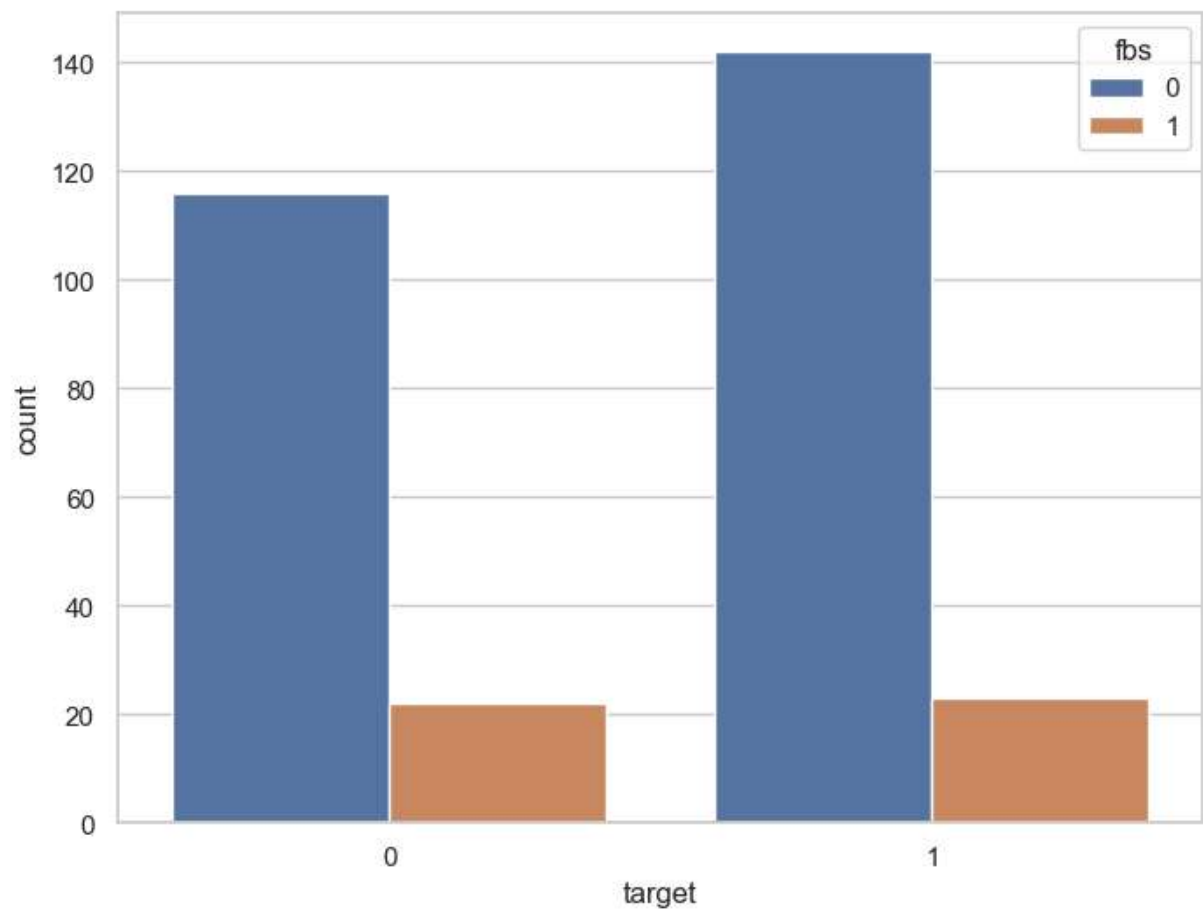
```
In [30]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="target", data=df, facecolor=(0, 1, 0, 1), linewidth=5, edgeco
plt.show()
```



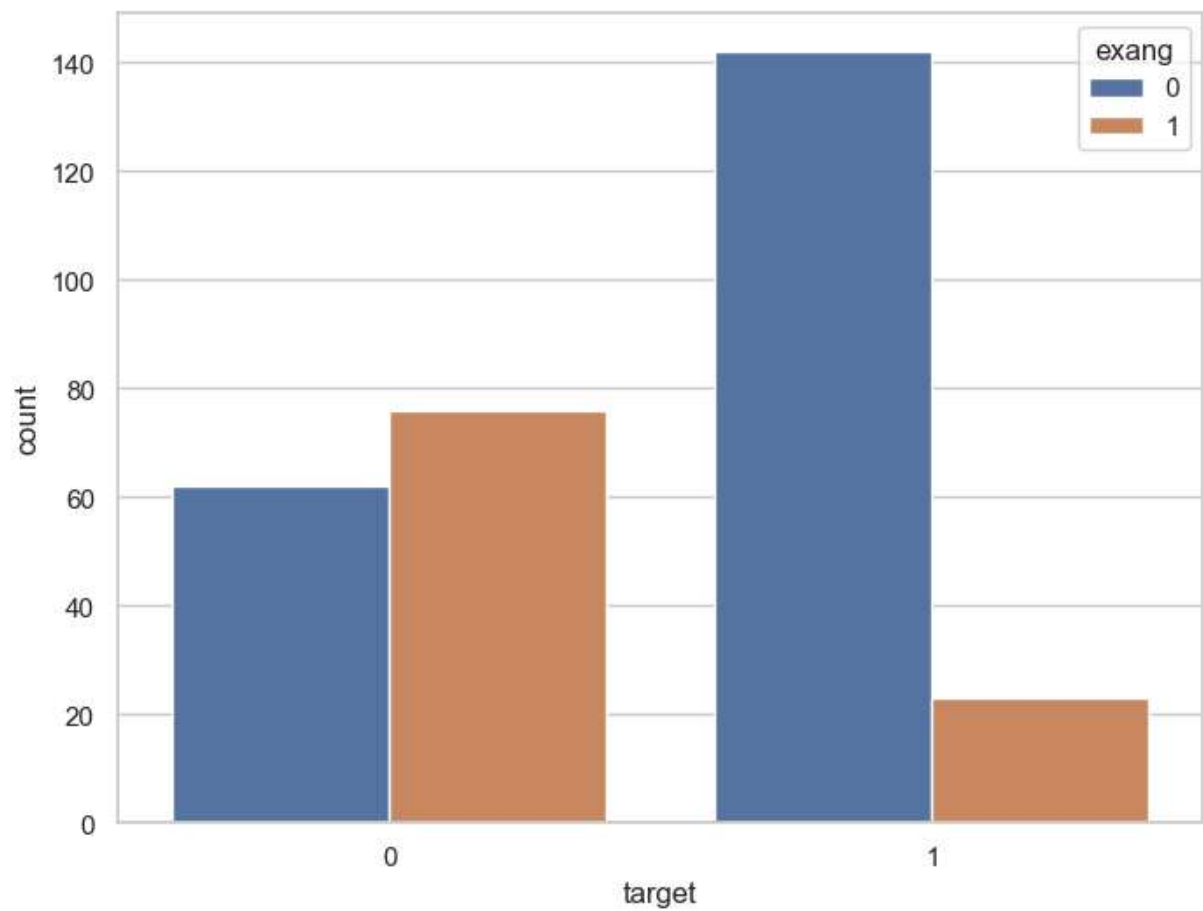
Comment

- I have visualize the `target` values distribution wrt `sex` .
- We can follow the same principles and visualize the `target` values distribution wrt `fbs` (fasting blood sugar) and `exang` (exercise induced angina)

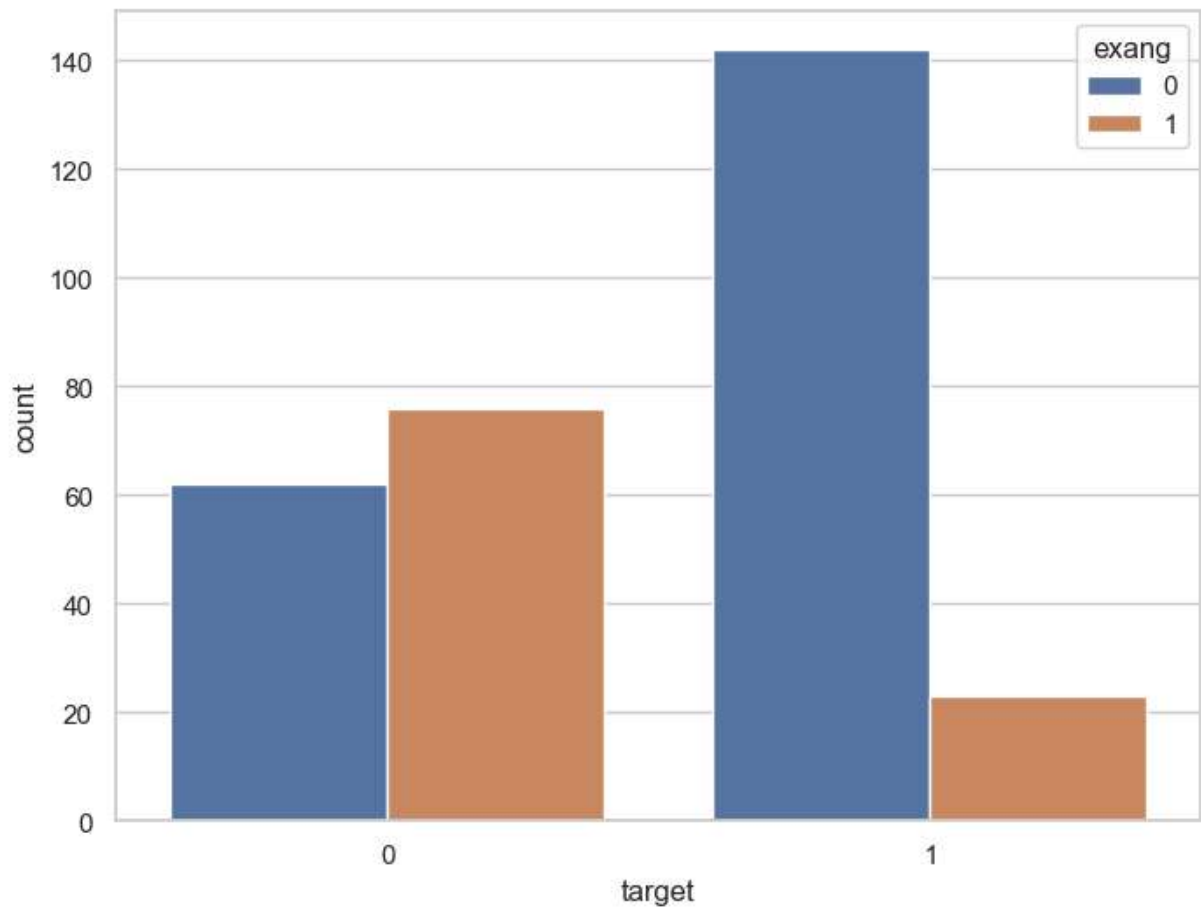
```
In [31]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="target", hue="fbs", data=df)
plt.show()
```



```
In [32]: f, ax = plt.subplots(figsize=(8, 6))  
ax = sns.countplot(x="target", hue="fbs", data=df)  
plt.show()
```



```
In [33]: plt.subplots(figsize=(8, 6))  
ax = sns.countplot(x="target", hue="exang", data=df)  
plt.show()
```



Bivariate Analysis

```
In [34]: correlation = df.corr()
```

```
In [35]: correlation['target'].sort_values(ascending=False)
```

```
Out[35]: target      1.000000  
cp          0.433798  
thalach     0.421741  
slope       0.345877  
restecg     0.137230  
fbs         -0.028046  
chol        -0.085239  
trestbps    -0.144931  
age         -0.225439  
sex         -0.280937  
thal        -0.344029  
ca          -0.391724  
oldpeak     -0.430696  
exang       -0.436757  
Name: target, dtype: float64
```

Explore cp variable

- `cp` stands for chest pain type.
- First, I will check number of unique values in `cp` variable.

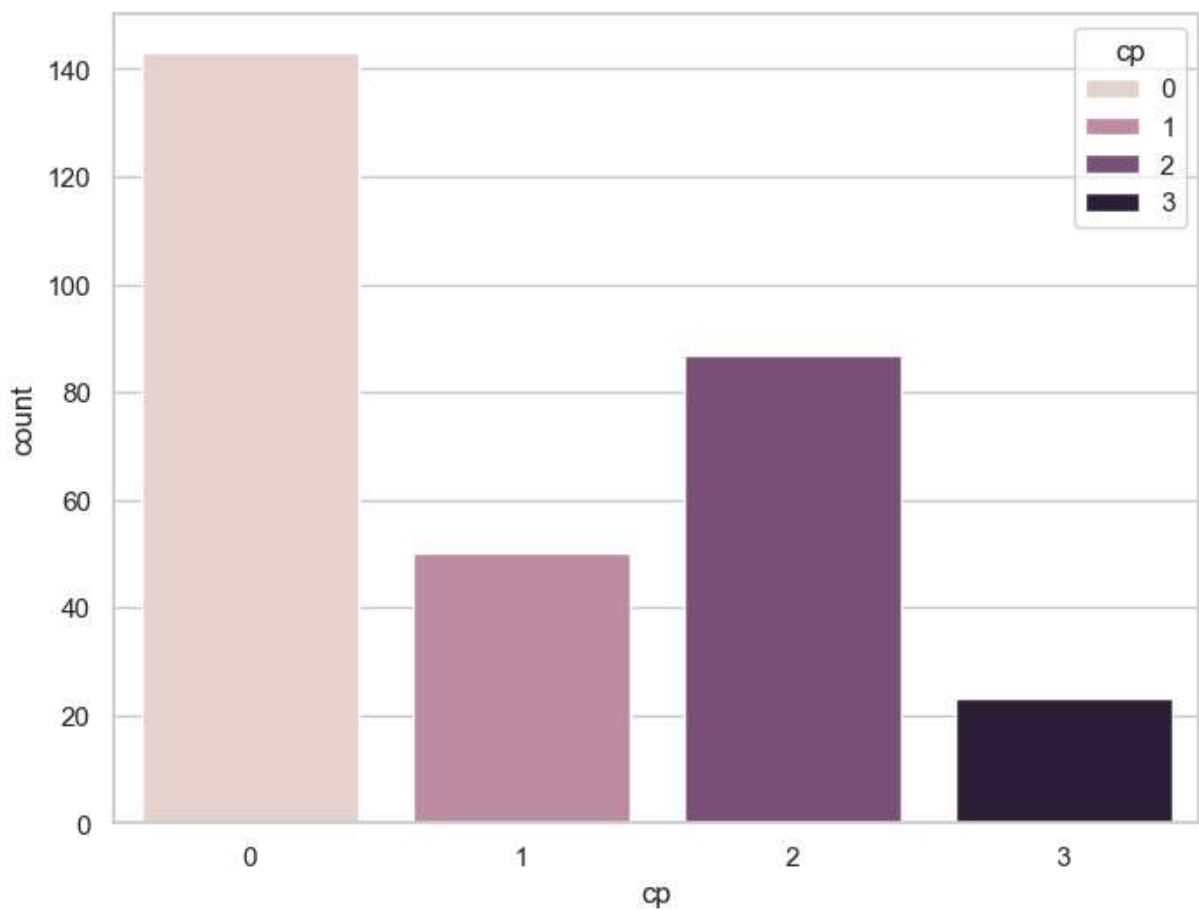
```
In [36]: df['cp'].nunique()
```

```
Out[36]: 4
```

```
In [37]: df['cp'].value_counts()
```

```
Out[37]: cp
0      143
2       87
1       50
3       23
Name: count, dtype: int64
```

```
In [42]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="cp", data=df, hue='cp')
plt.show()
```

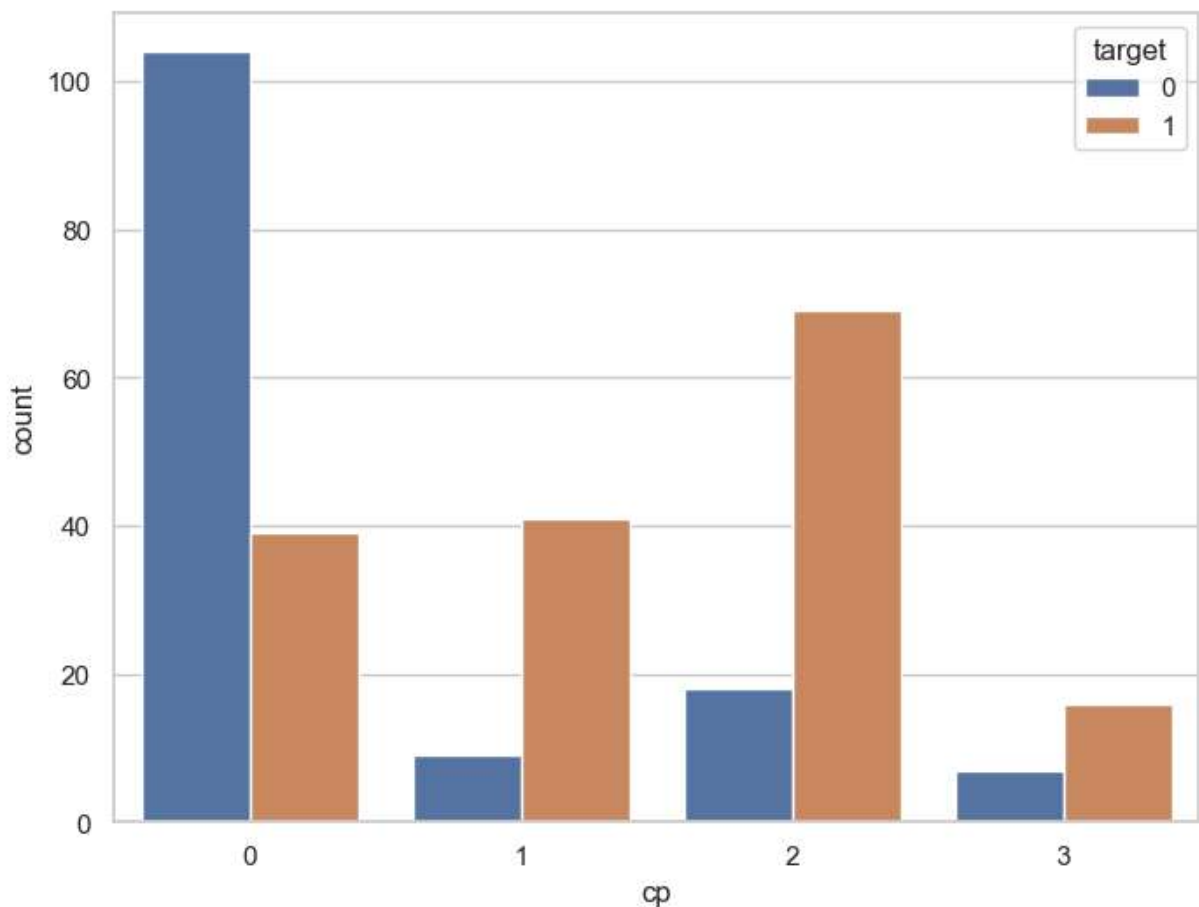


```
In [43]: df.groupby('cp')['target'].value_counts()
```



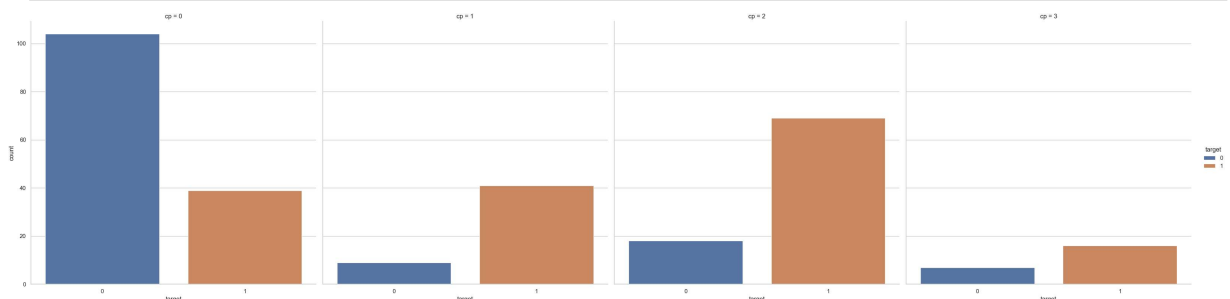
```
Out[43]: cp target
0 0      104
    1      39
1 1      41
    0       9
2 1      69
    0      18
3 1      16
    0       7
Name: count, dtype: int64
```

```
In [44]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.countplot(x="cp", hue="target", data=df)
plt.show()
```



```
In [51]: ax = sns.catplot(x="target", col="cp", data=df, kind="count", height=8, aspect=1, hu
```

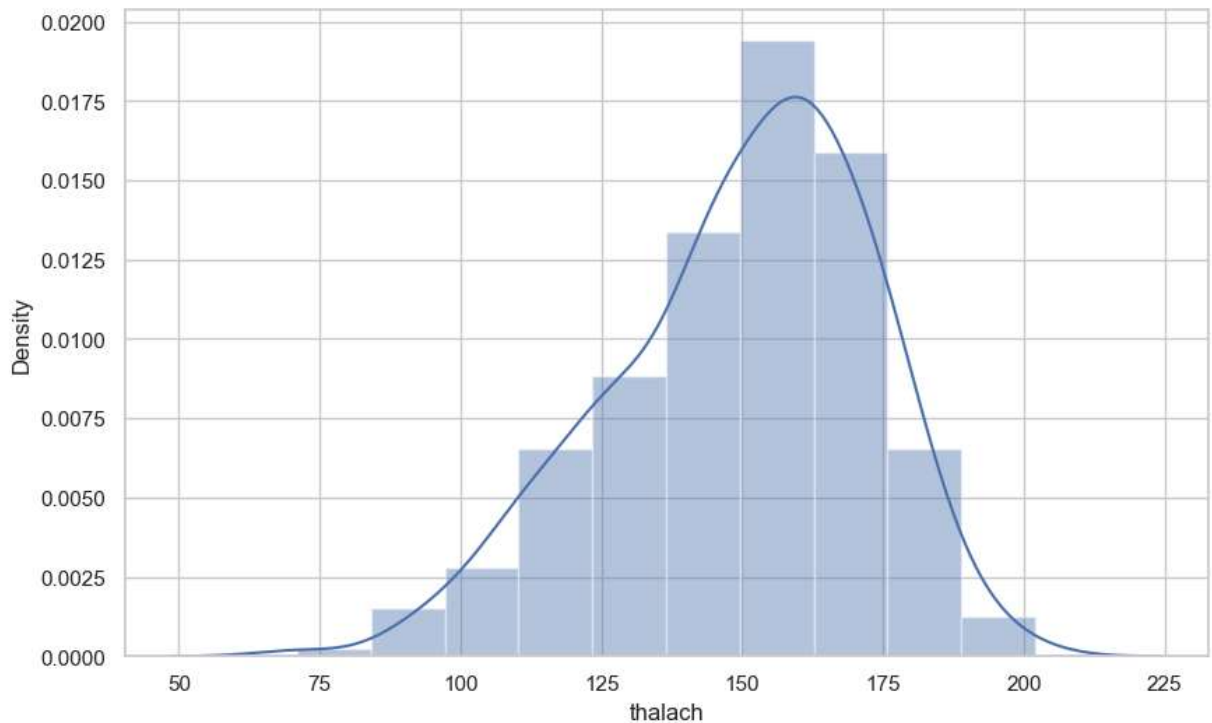
```
In [52]: plt.show(ax)
```



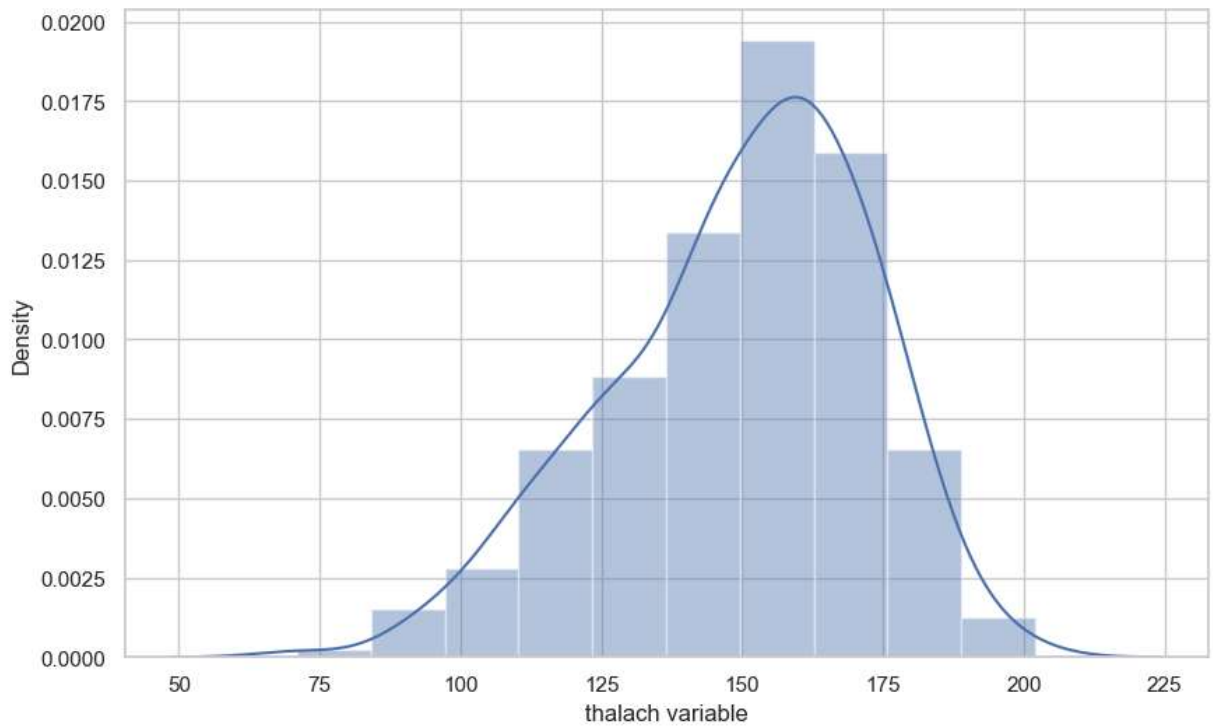
Explore thalach variable

- `thalach` stands for maximum heart rate achieved.
- I will check number of unique values in `thalach` variable as follows :

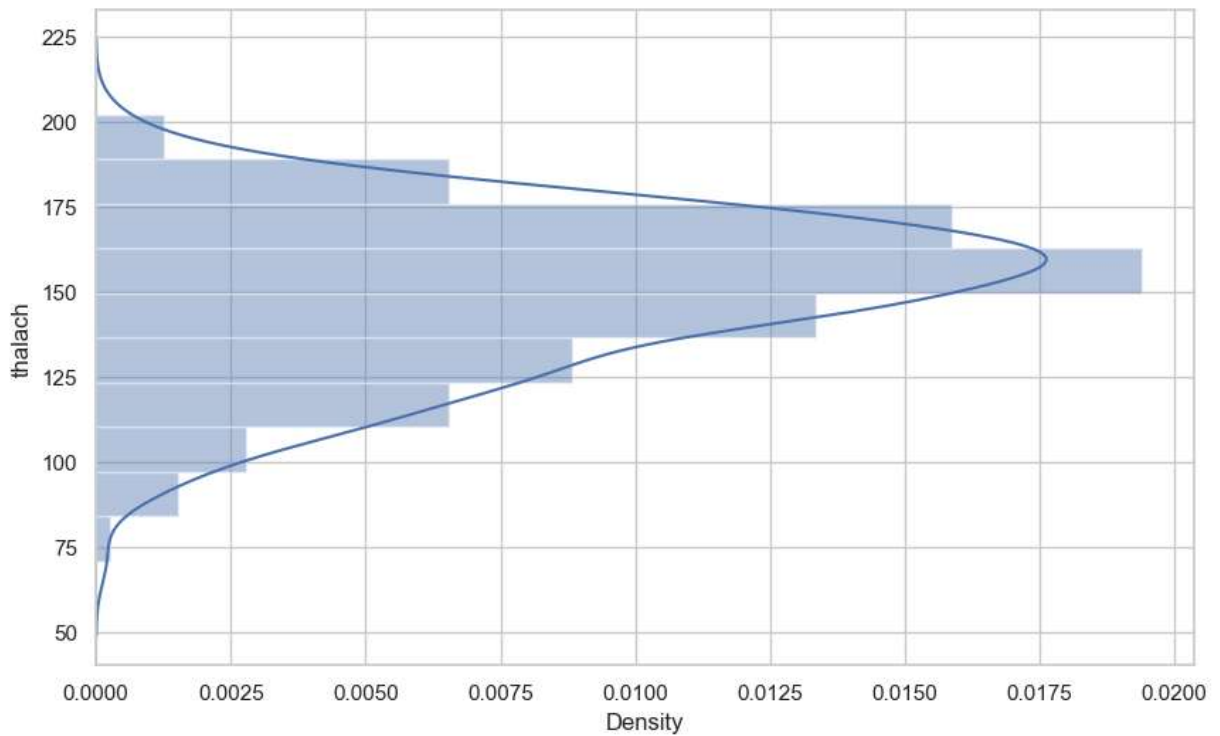
```
In [53]: f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
ax = sns.distplot(x, bins=10)
plt.show()
```



```
In [54]: f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.distplot(x, bins=10)
plt.show()
```



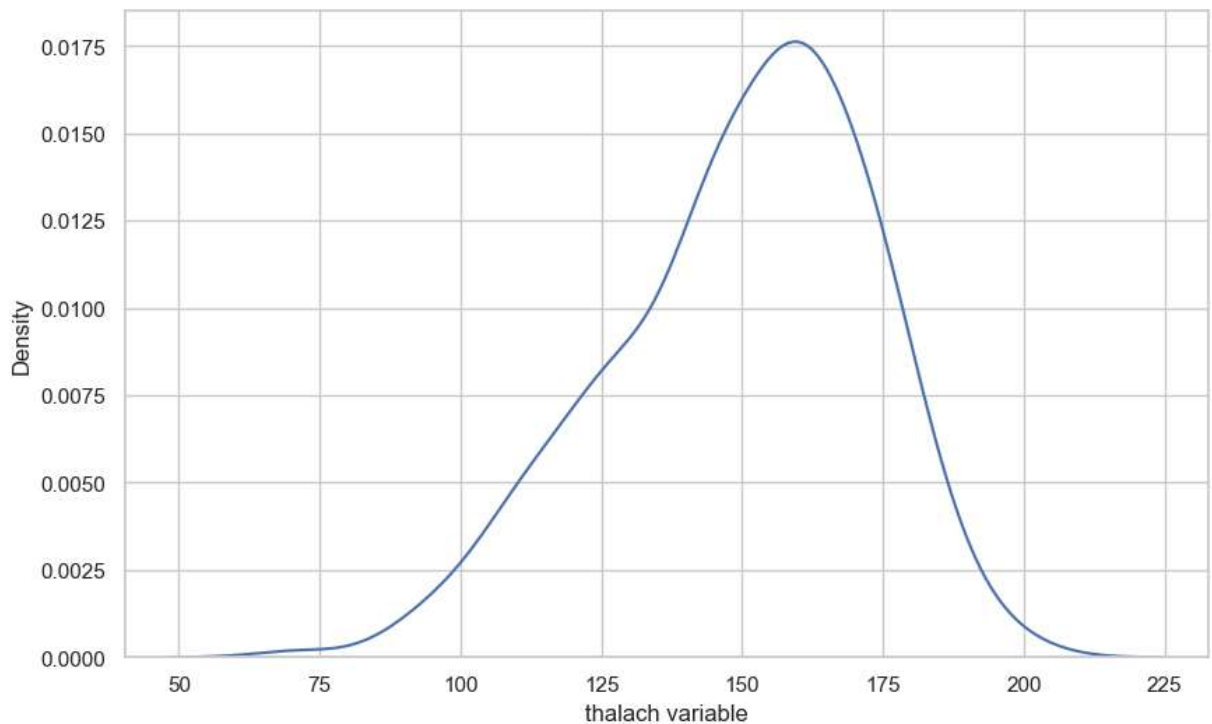
```
In [55]: f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
ax = sns.distplot(x, bins=10, vertical=True)
plt.show()
```



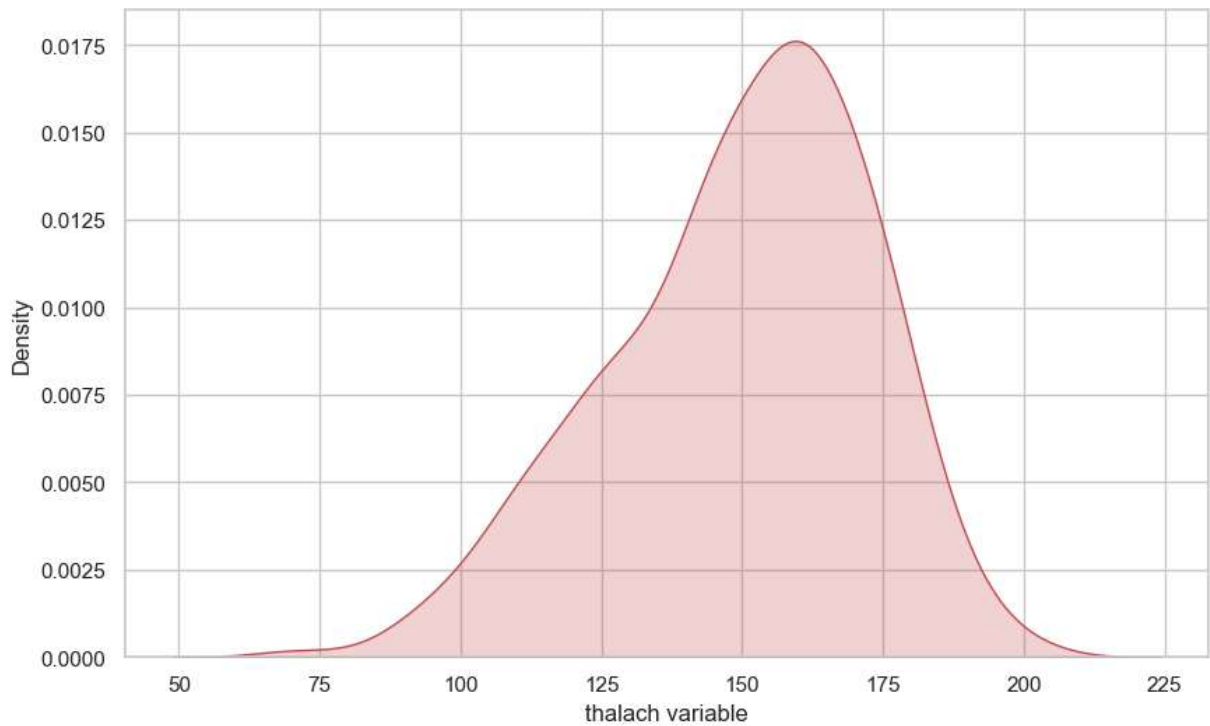
Seaborn Kernel Density Estimation (KDE) Plot

- The kernel density estimate (KDE) plot is a useful tool for plotting the shape of a distribution.
- The KDE plot plots the density of observations on one axis with height along the other axis.
- We can plot a KDE plot as follows :

```
In [56]: f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.kdeplot(x)
plt.show()
```



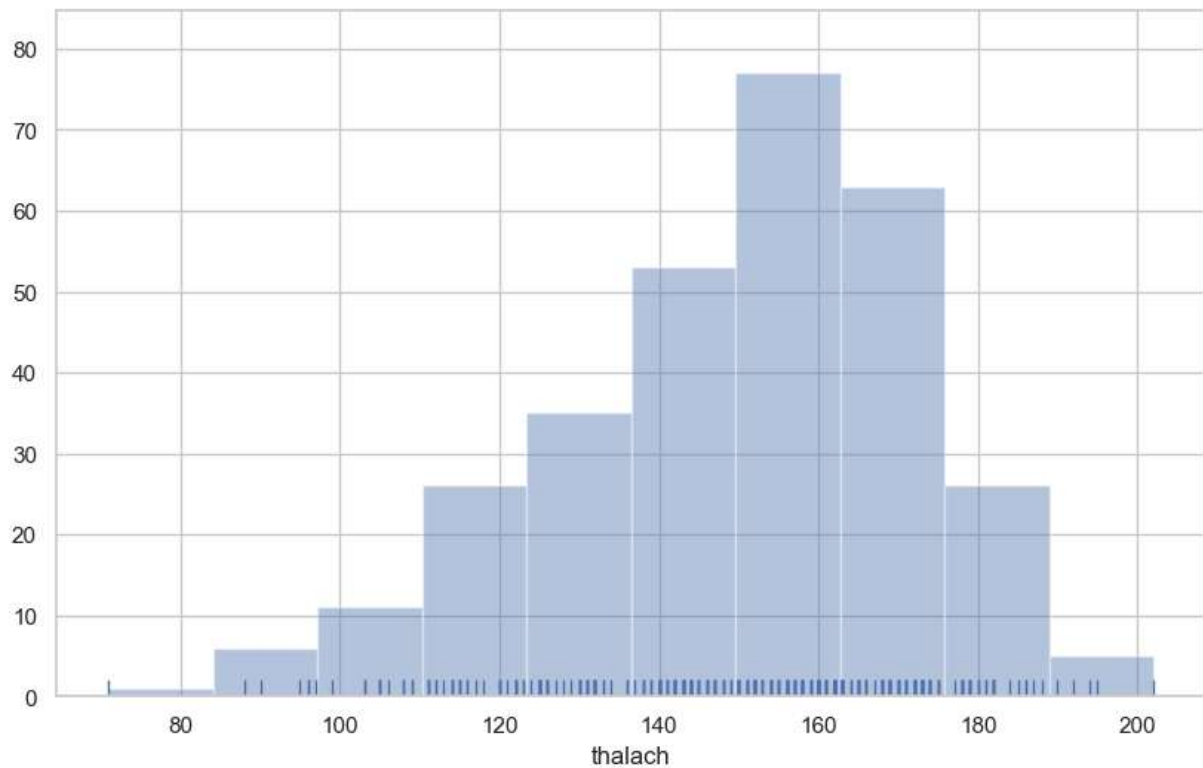
```
In [57]: f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
x = pd.Series(x, name="thalach variable")
ax = sns.kdeplot(x, shade=True, color='r')
plt.show()
```



Histogram

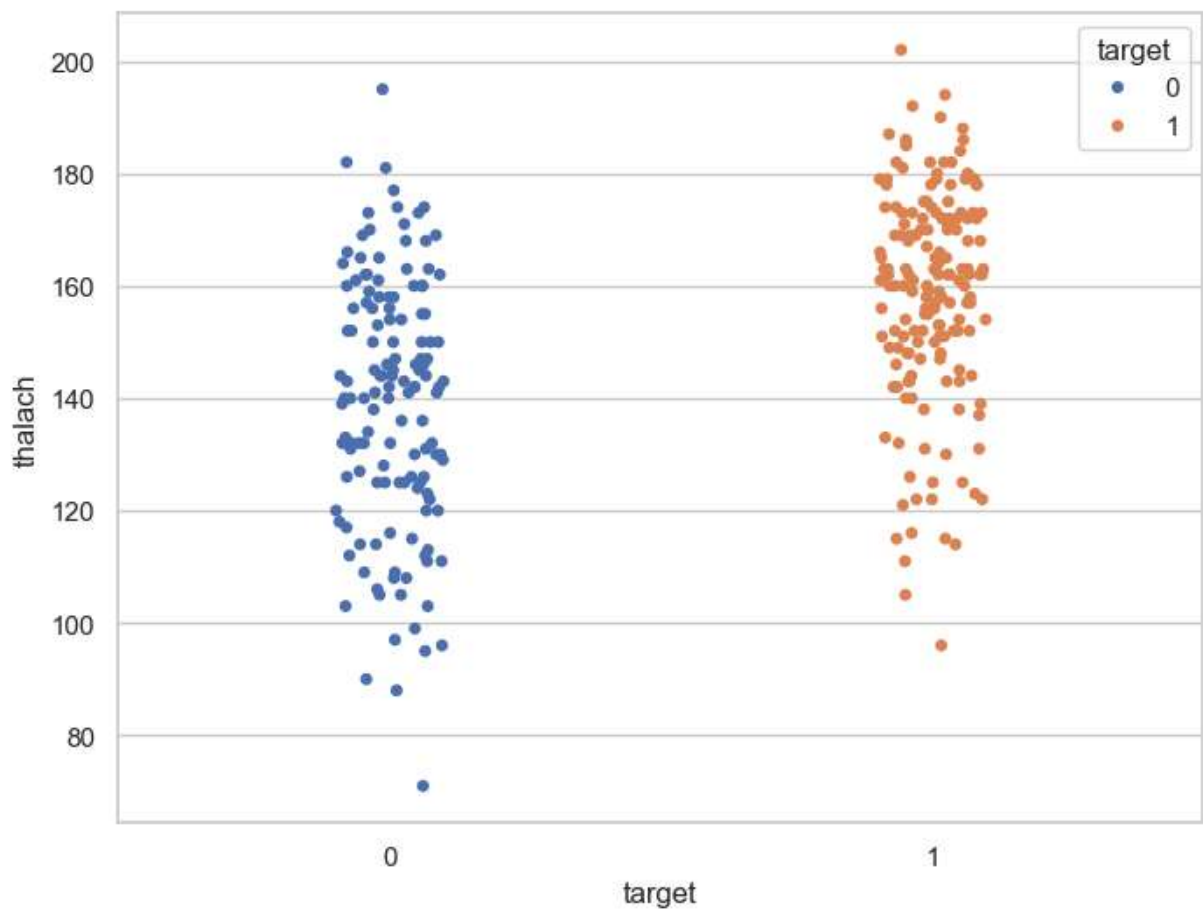
- A histogram represents the distribution of data by forming bins along the range of the data and then drawing bars to show the number of observations that fall in each bin.
- We can plot a histogram as follows :

```
In [58]: f, ax = plt.subplots(figsize=(10,6))
x = df['thalach']
ax = sns.distplot(x, kde=False, rug=True, bins=10)
plt.show()
```



Visualize frequency distribution of thalach variable wrt target

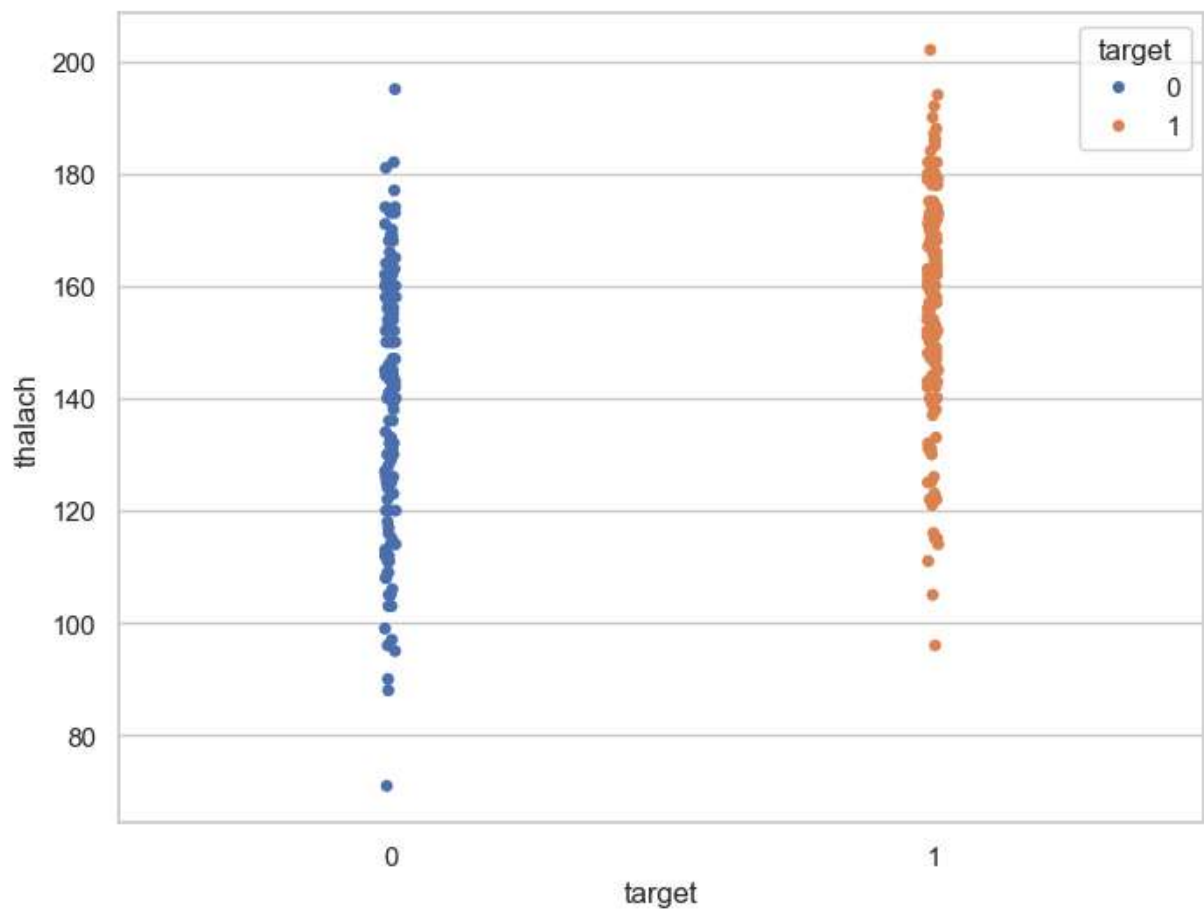
```
In [61]: f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="target", y="thalach", data=df, hue='target')
plt.show()
```



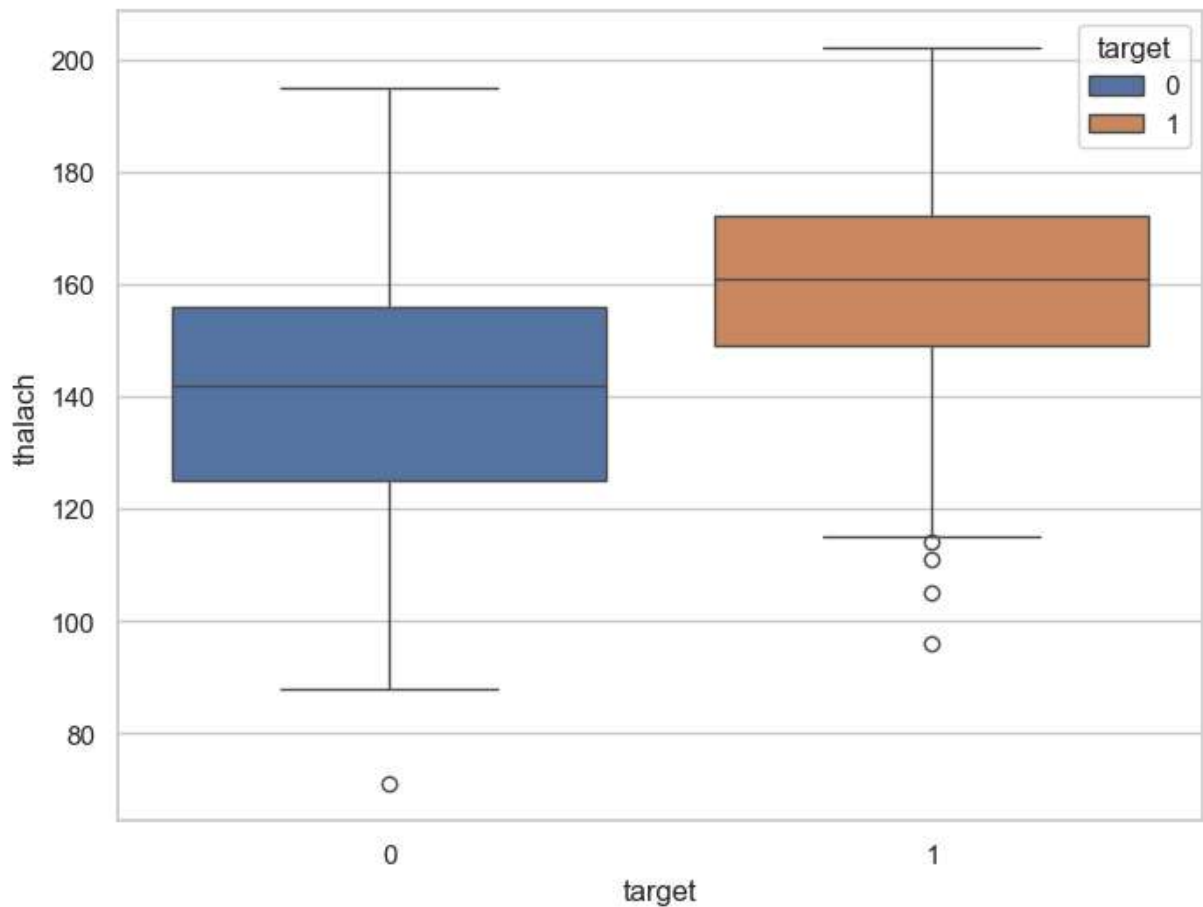
Interpretation

- We can see that those people suffering from heart disease (target = 1) have relatively higher heart rate (thalach) as compared to people who are not suffering from heart disease (target = 0).

```
In [63]: f, ax = plt.subplots(figsize=(8, 6))
sns.stripplot(x="target", y="thalach", data=df, jitter = 0.01, hue='target')
plt.show()
```



```
In [65]: f, ax = plt.subplots(figsize=(8, 6))
sns.boxplot(x="target", y="thalach", data=df, hue='target')
plt.show()
```

9. Multivariate analysis

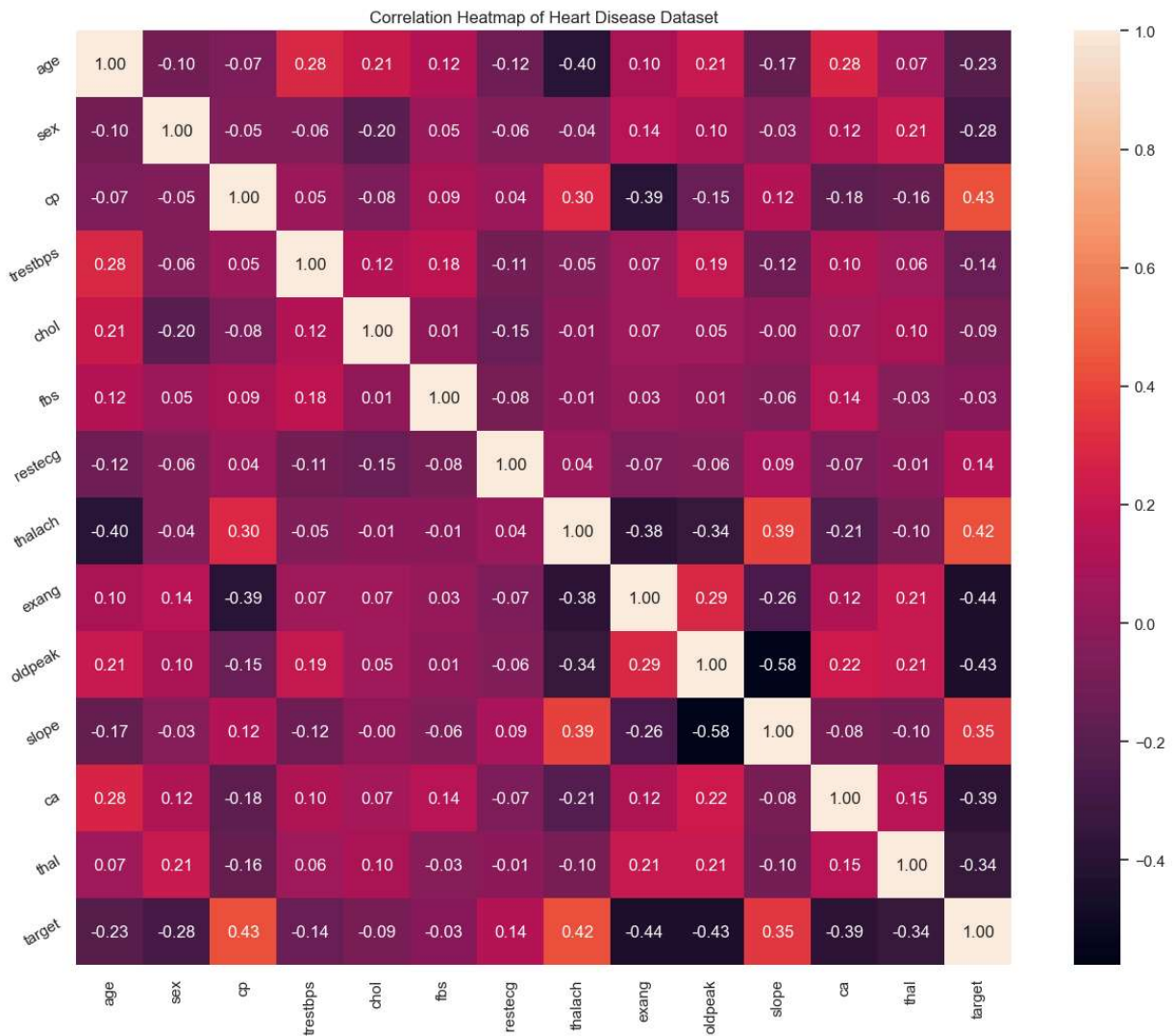
- The objective of the multivariate analysis is to discover patterns and relationships in the dataset.

Discover patterns and relationships

- An important step in EDA is to discover patterns and relationships between variables in the dataset.
- I will use `heat map` and `pair plot` to discover the patterns and relationships in the dataset.
- First of all, I will draw a `heat map`.

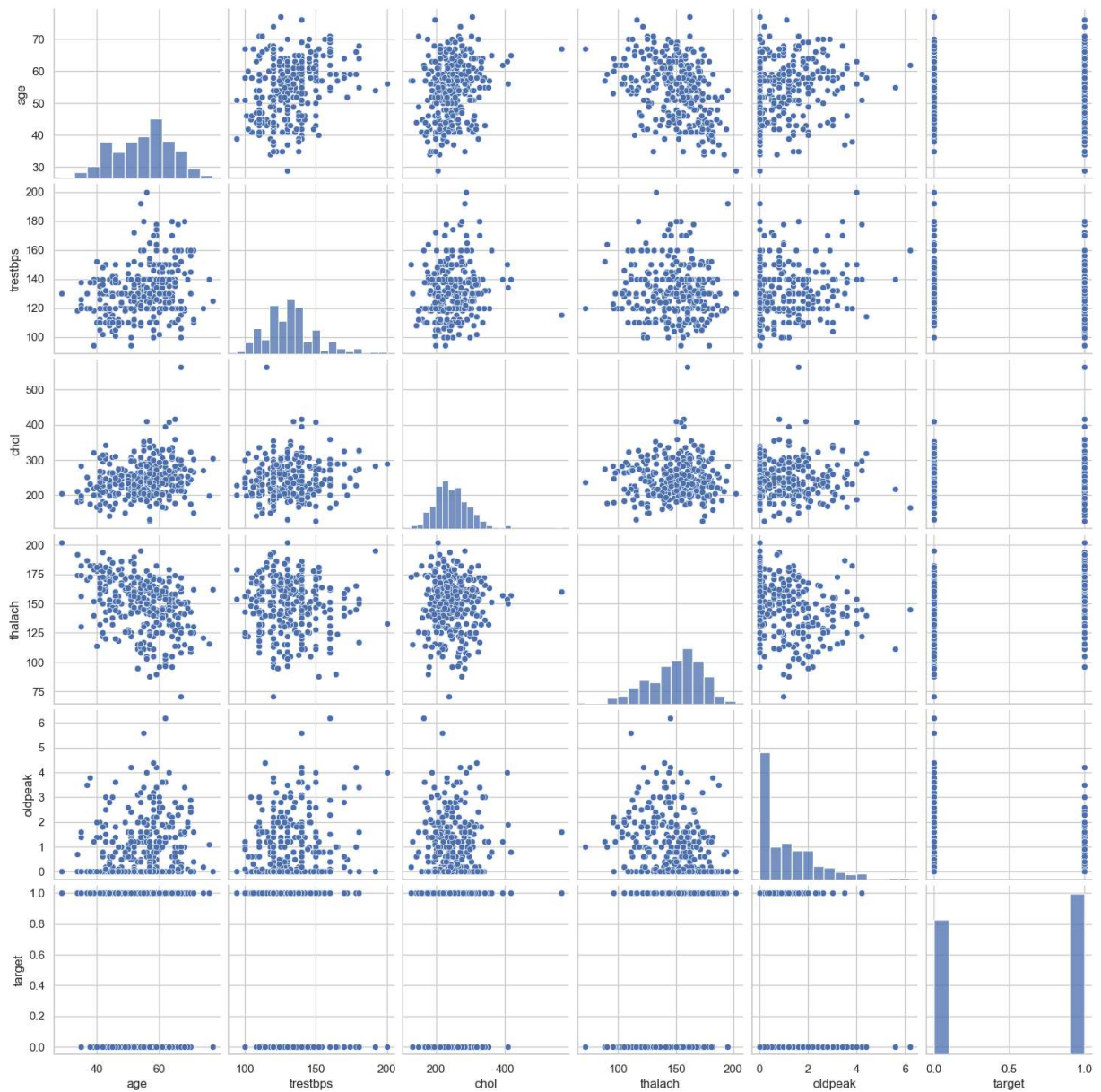
Heatmap

```
In [66]: plt.figure(figsize=(16,12))
plt.title('Correlation Heatmap of Heart Disease Dataset')
a = sns.heatmap(correlation, square=True, annot=True, fmt='.2f', linecolor='white')
a.set_xticklabels(a.get_xticklabels(), rotation=90)
a.set_yticklabels(a.get_yticklabels(), rotation=30)
plt.show()
```



Pair plot

```
In [67]: num_var = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak', 'target']
sns.pairplot(df[num_var], kind='scatter', diag_kind='hist')
plt.show()
```



```
In [68]: df['age'].nunique()
```

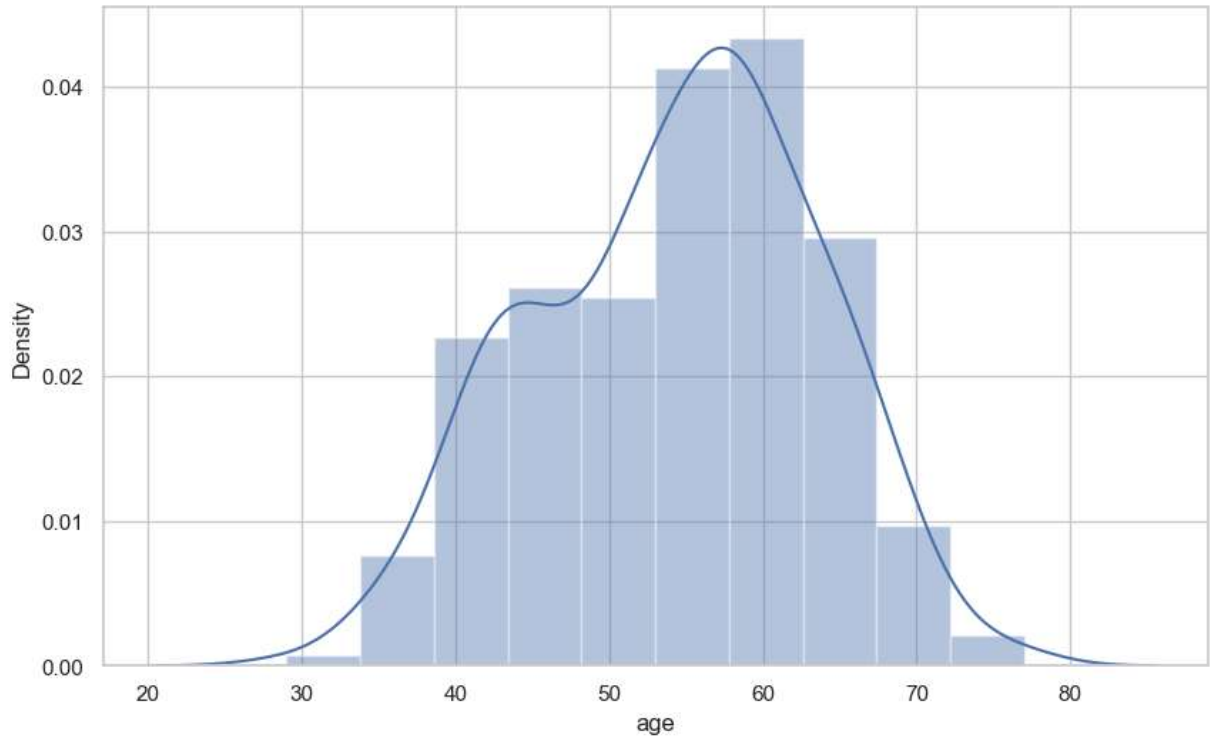
```
Out[68]: 41
```

```
In [70]: df['age'].describe()
```

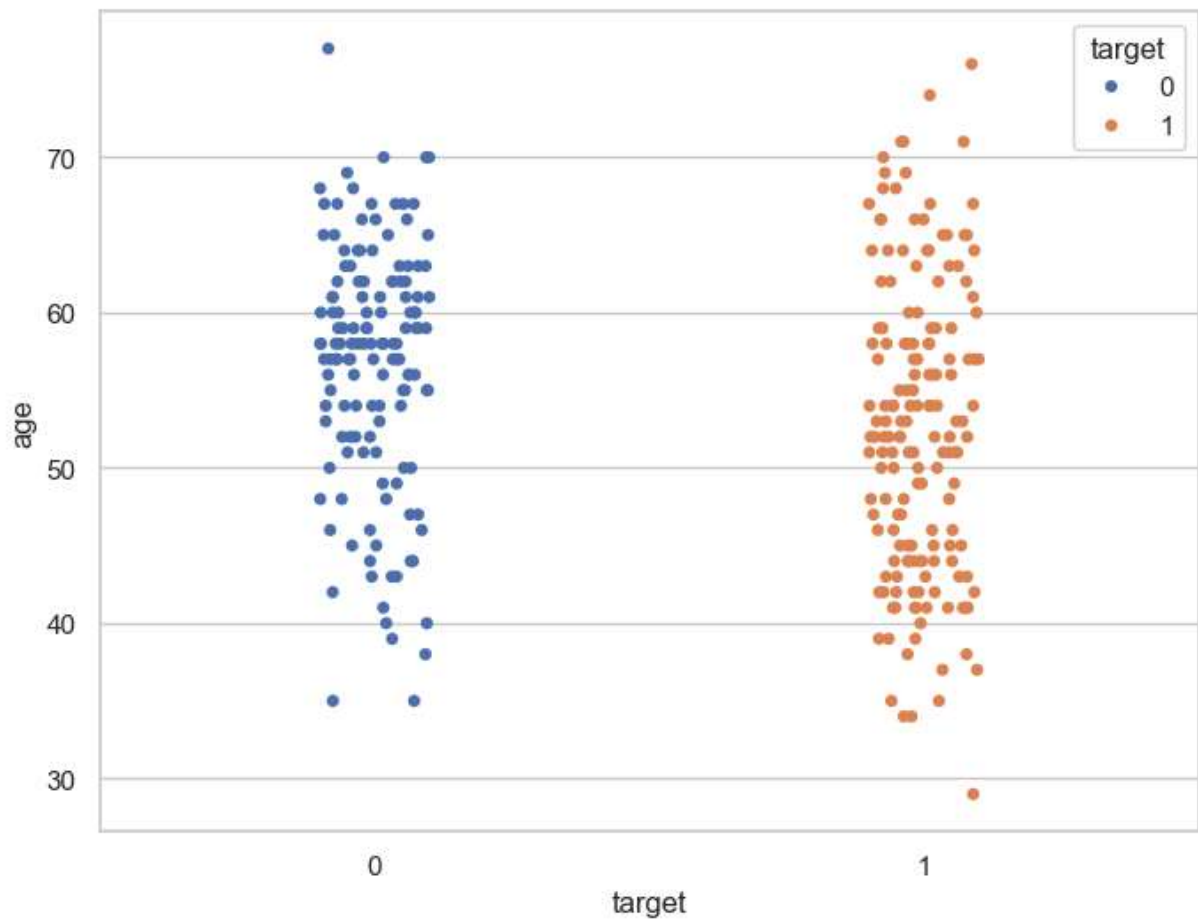
```
Out[70]: count    303.000000
mean       54.366337
std        9.082101
min        29.000000
25%        47.500000
50%        55.000000
75%        61.000000
max        77.000000
Name: age, dtype: float64
```

```
In [71]: f, ax = plt.subplots(figsize=(10,6))
x = df['age']
```

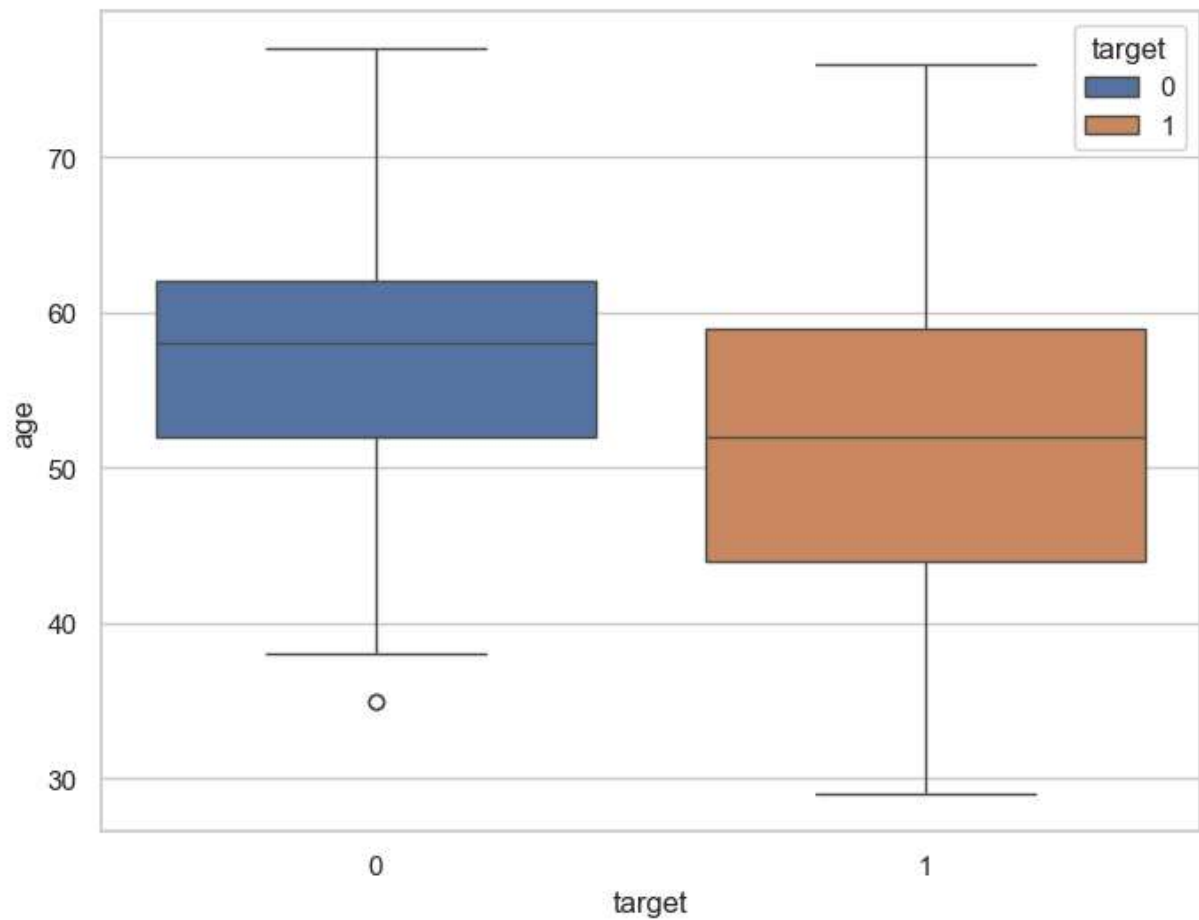
```
ax = sns.distplot(x, bins=10)  
plt.show()
```



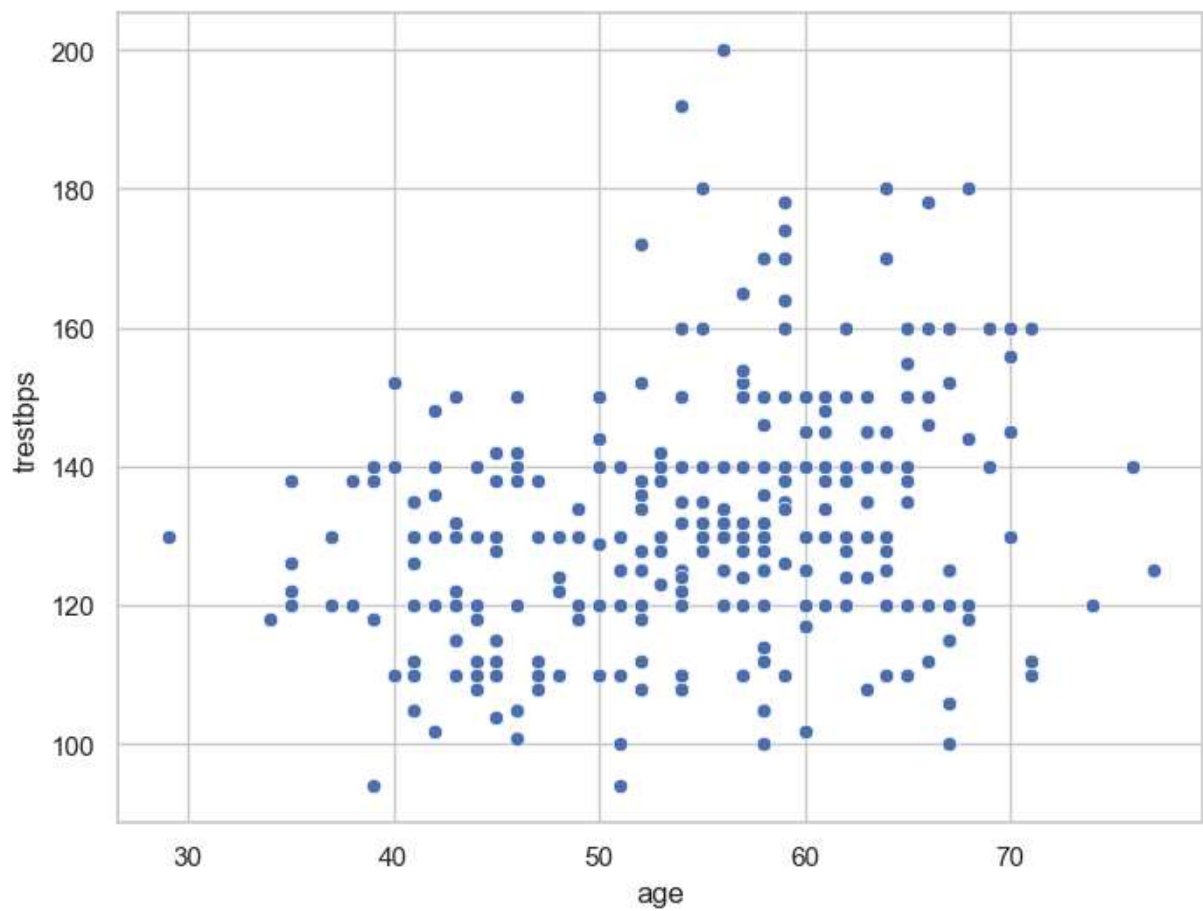
```
In [73]: f, ax = plt.subplots(figsize=(8, 6))  
sns.stripplot(x="target", y="age", data=df, hue='target')  
plt.show()
```



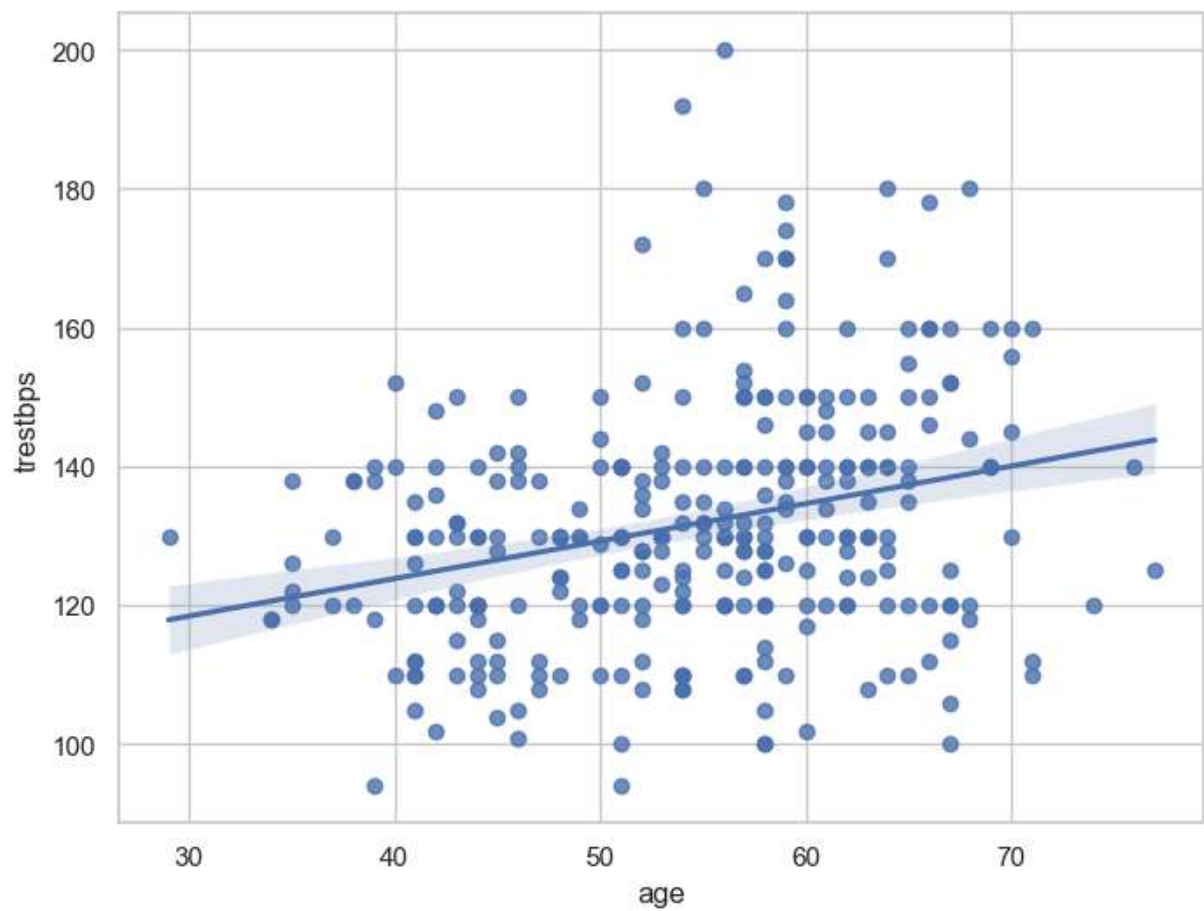
```
In [74]: f, ax = plt.subplots(figsize=(8, 6))  
sns.boxplot(x="target", y="age", data=df, hue='target')  
plt.show()
```



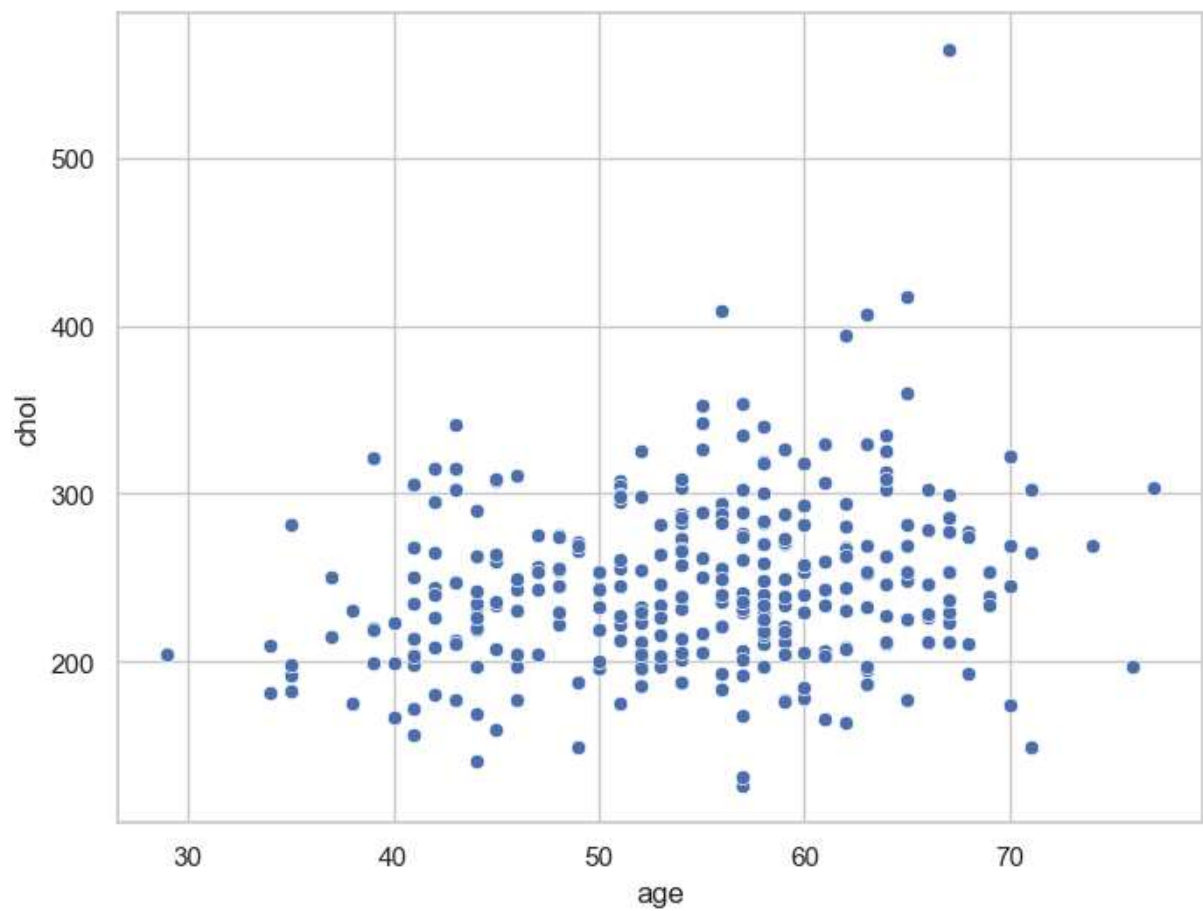
```
In [75]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.scatterplot(x="age", y="trestbps", data=df)
plt.show()
```



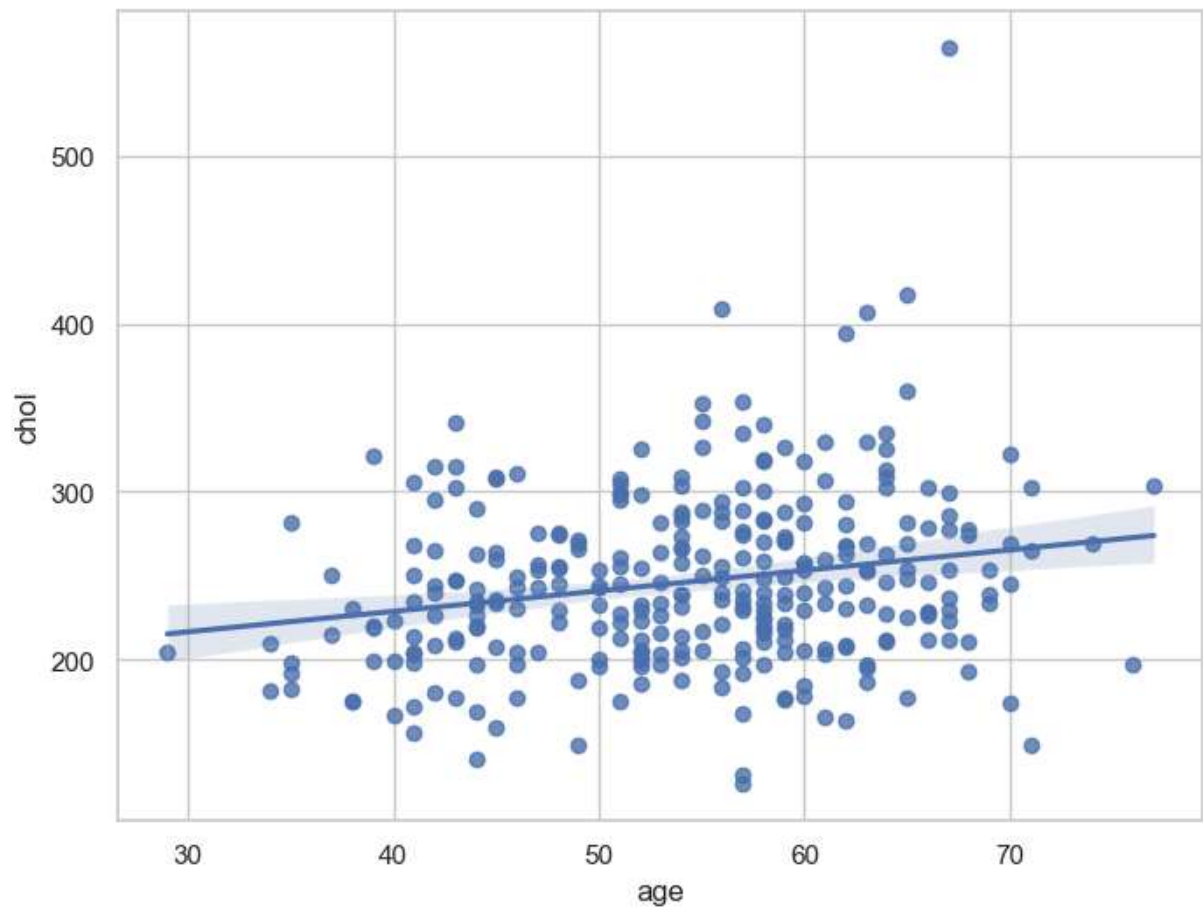
```
In [76]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="age", y="trestbps", data=df)
plt.show()
```



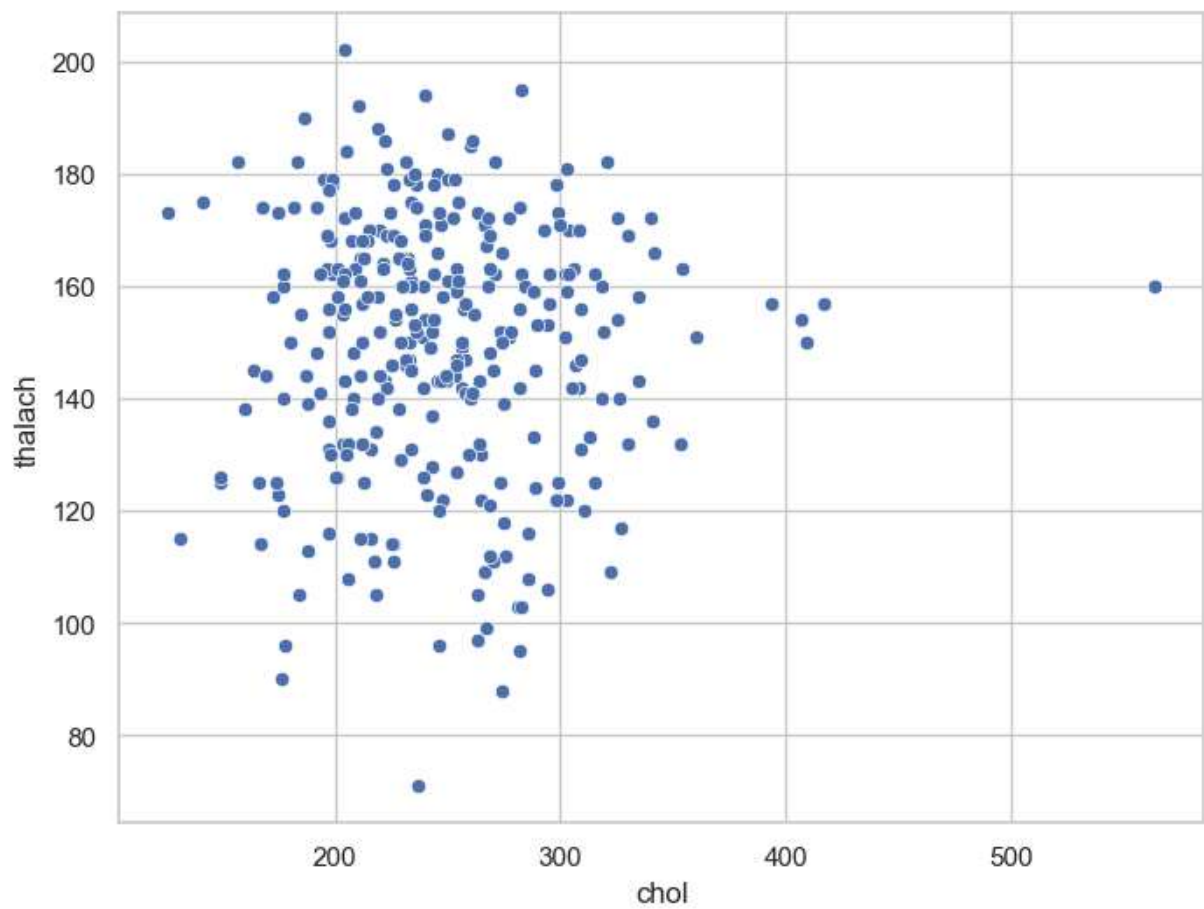
```
In [77]: f, ax = plt.subplots(figsize=(8, 6))  
ax = sns.scatterplot(x="age", y="chol", data=df)  
plt.show()
```

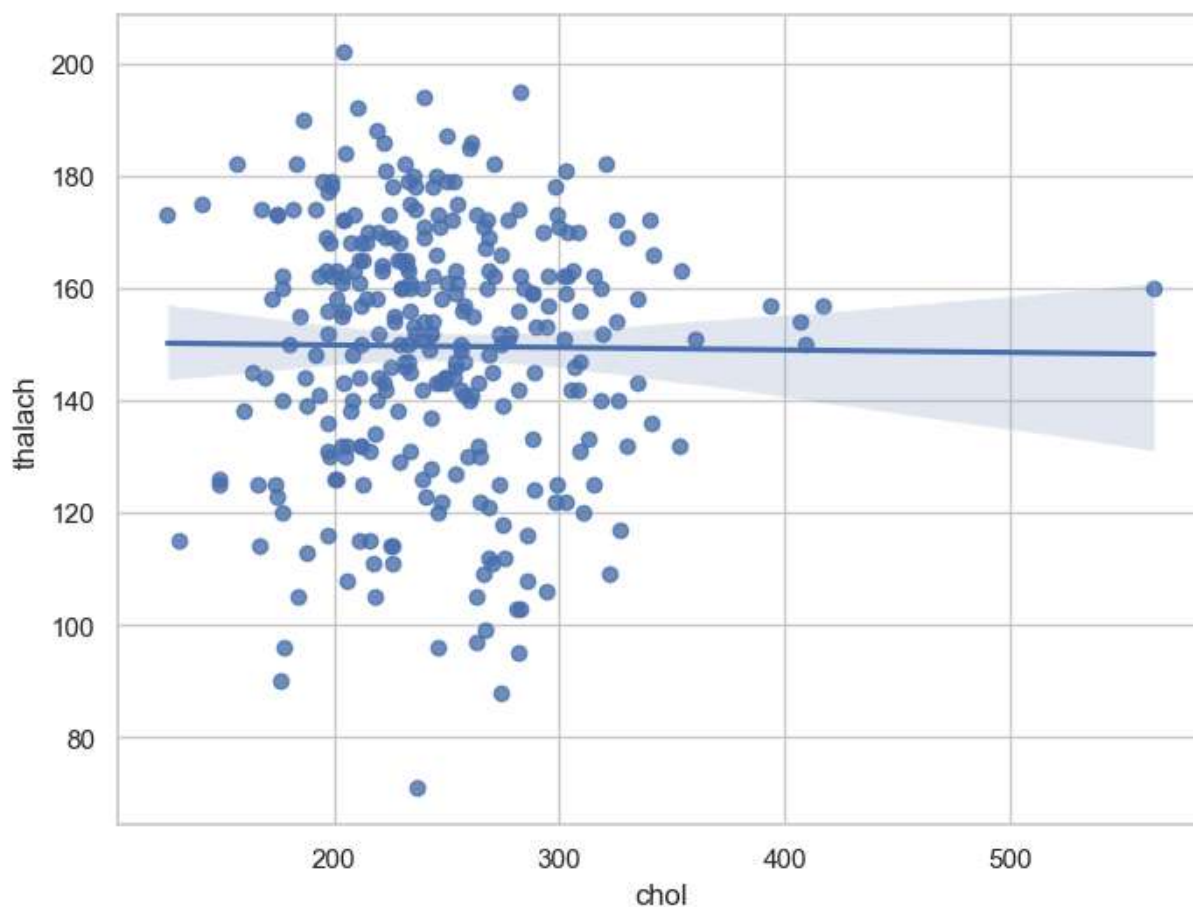
```
In [78]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="age", y="chol", data=df)
plt.show()
```



```
In [79]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.scatterplot(x="chol", y = "thalach", data=df)
plt.show()
```



```
In [80]: f, ax = plt.subplots(figsize=(8, 6))
ax = sns.regplot(x="chol", y="thalach", data=df)
plt.show()
```



10. Dealing with missing values - In Pandas missing data is represented by two values: - **None**: None is a Python singleton object that is often used for missing data in Python code. - **NaN**: NaN (an acronym for Not a Number), is a special floating-point value recognized by all systems that use the standard IEEE floating-point representation. - There are different methods in place on how to detect missing values. **### Pandas isnull() and notnull() functions** - Pandas offers two functions to test for missing data - `isnull()` and `notnull()`. These are simple functions that return a boolean value indicating whether the passed in argument value is in fact missing data. - Below, I will list some useful commands to deal with missing values. **### Useful commands to detect missing values** - **`df.isnull()`** The above command checks whether each cell in a dataframe contains missing values or not. If the cell contains missing value, it returns True otherwise it returns False. - **`df.isnull().sum()`** The above command returns total number of missing values in each column in the dataframe. - **`df.isnull().sum().sum()`** It returns total number of missing values in the dataframe. - **`df.isnull().mean()`** It returns percentage of missing values in each column in the dataframe. - **`df.isnull().any()`** It checks which column has null values and which has not. The columns which has null values returns TRUE and FALSE otherwise. - **`df.isnull().any().any()`** It returns a boolean value indicating whether the dataframe has missing values or not. If dataframe contains missing values it returns TRUE and FALSE otherwise. - **`df.isnull().values.any()`** It checks whether a particular column has missing values or not. If the column contains missing values, then it returns TRUE otherwise FALSE. - **`df.isnull().values.sum()`** It returns the total number of missing values in the dataframe.

```
In [88]: df.isnull().sum()
```

```
Out[88]: age      0
        sex      0
        cp       0
        trestbps  0
        chol     0
        fbs      0
        restecg   0
        thalach   0
        exang     0
        oldpeak   0
        slope     0
        ca        0
        thal      0
        target    0
        dtype: int64
```

11. Check with ASSERT statement - Back to Table of Contents. - We must confirm that our dataset has no missing values. - We can write an **assert statement** to verify this. - We can use an assert statement to programmatically check that no missing, unexpected 0 or negative values are present. - This gives us confidence that our code is running properly. - **Assert statement** will return nothing if the value being tested is true and will throw an AssertionError if the value is false. - **Asserts** - `assert 1 == 1` (return Nothing if the value is True) - `assert 1 == 2` (return AssertionError if the value is False)

```
In [96]: assert pd.notnull(df).all().all()
```

```
In [97]: assert (df >= 0).all().all()
```

12.Outlier detection

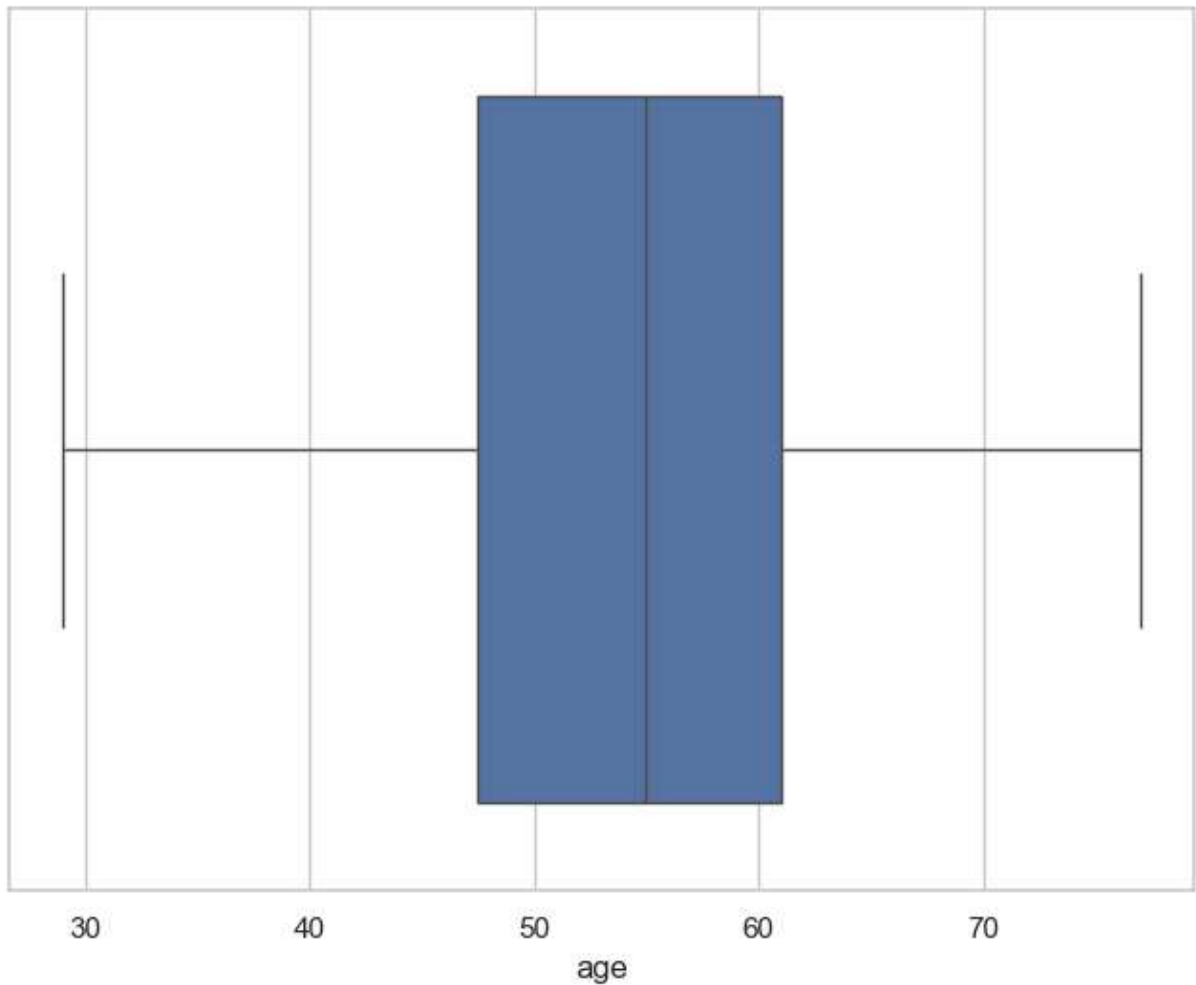
I will make boxplots to visualise outliers in the continuous numerical variables : -

age, trestbps, chol, thalach and oldpeak variables.

```
In [98]: df['age'].describe()
```

```
Out[98]: count    303.000000
        mean     54.366337
        std       9.082101
        min      29.000000
        25%      47.500000
        50%      55.000000
        75%      61.000000
        max      77.000000
        Name: age, dtype: float64
```

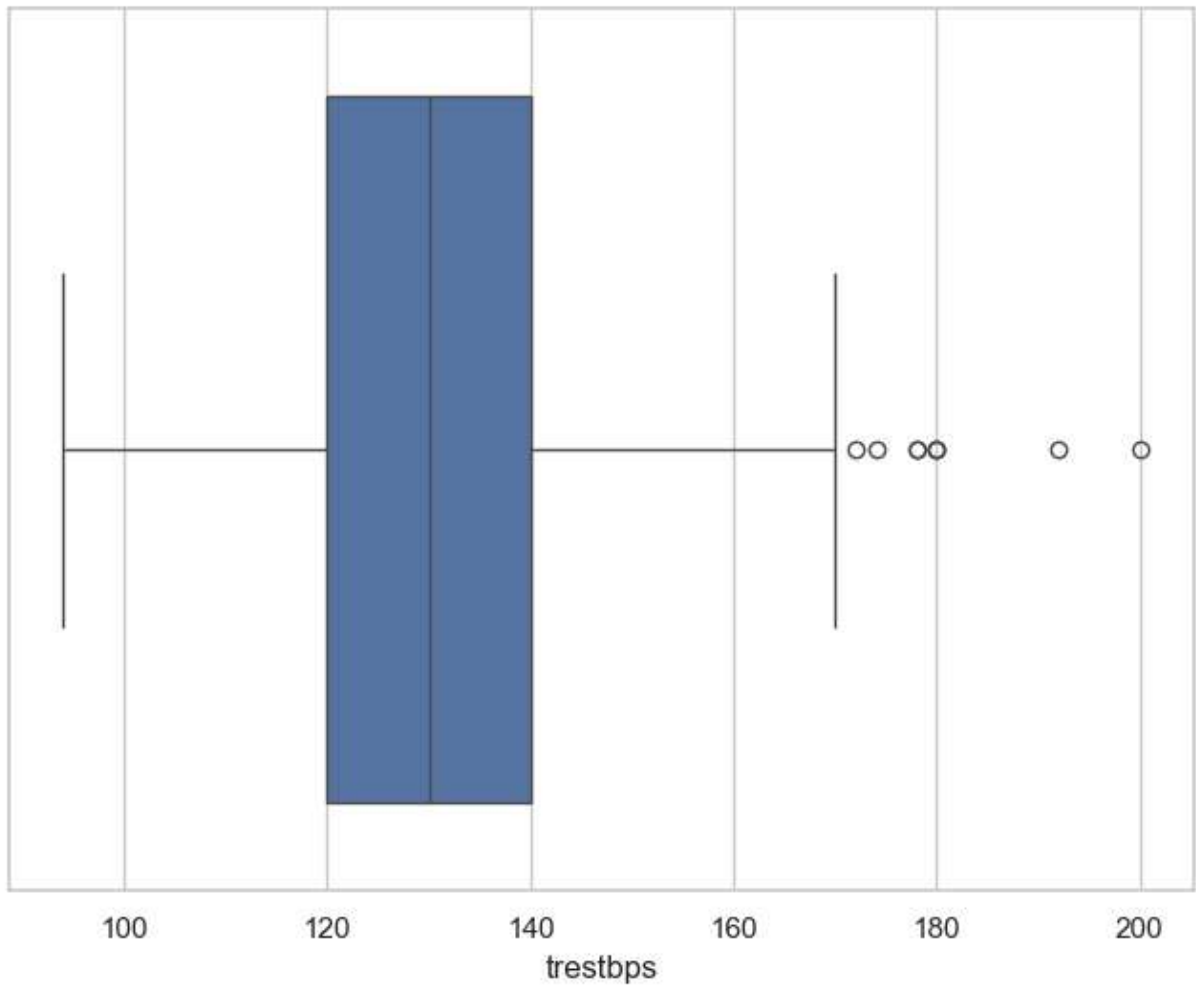
```
In [99]: f, ax = plt.subplots(figsize=(8, 6))
        sns.boxplot(x=df["age"])
        plt.show()
```



```
In [100...] df['trestbps'].describe()
```

```
Out[100...] count    303.000000  
mean      131.623762  
std       17.538143  
min       94.000000  
25%      120.000000  
50%      130.000000  
75%      140.000000  
max       200.000000  
Name: trestbps, dtype: float64
```

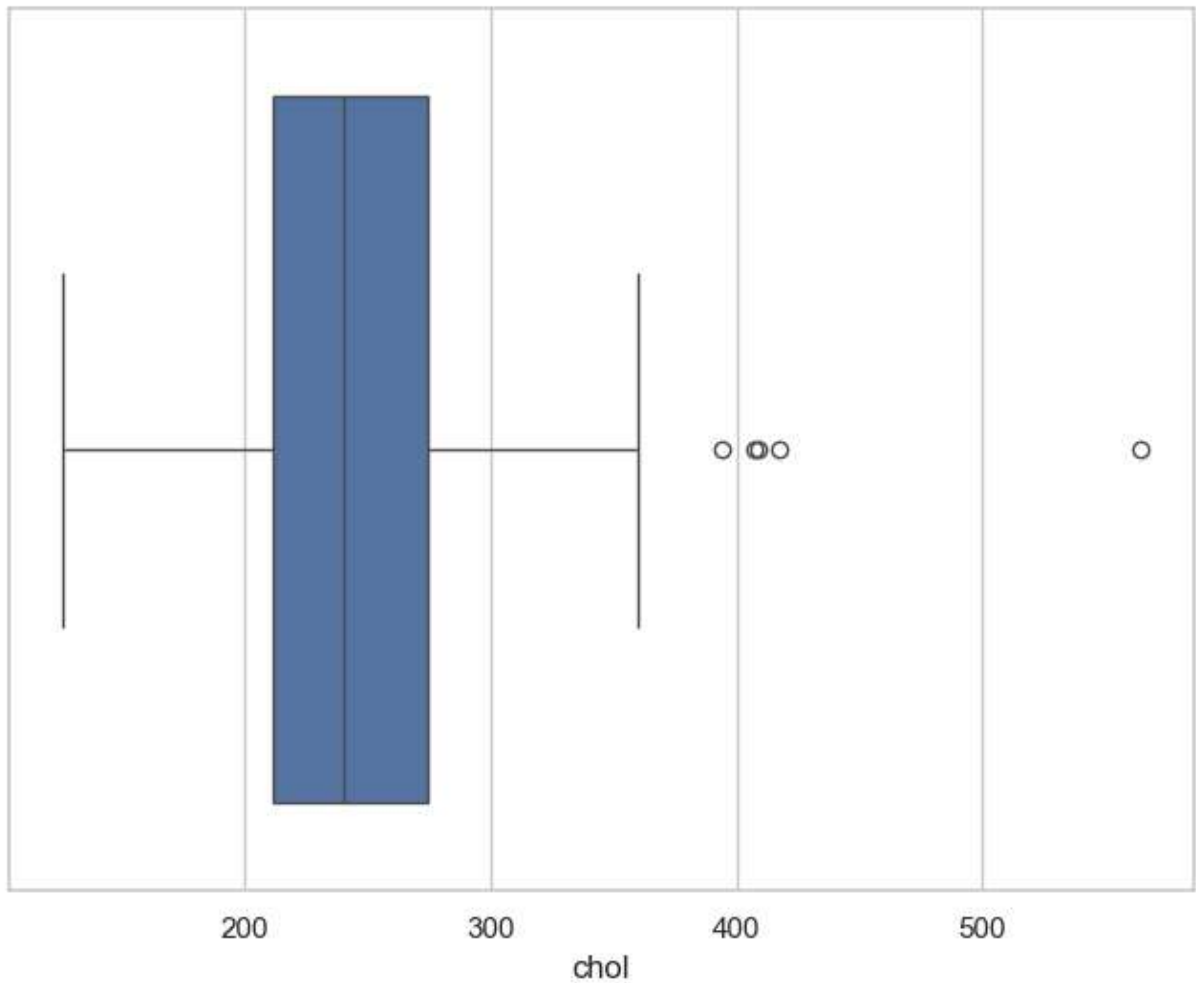
```
In [101...] f, ax = plt.subplots(figsize=(8, 6))  
sns.boxplot(x=df["trestbps"])  
plt.show()
```



```
In [102...] df['chol'].describe()
```

```
Out[102...] count    303.000000
mean      246.264026
std       51.830751
min       126.000000
25%       211.000000
50%       240.000000
75%       274.500000
max       564.000000
Name: chol, dtype: float64
```

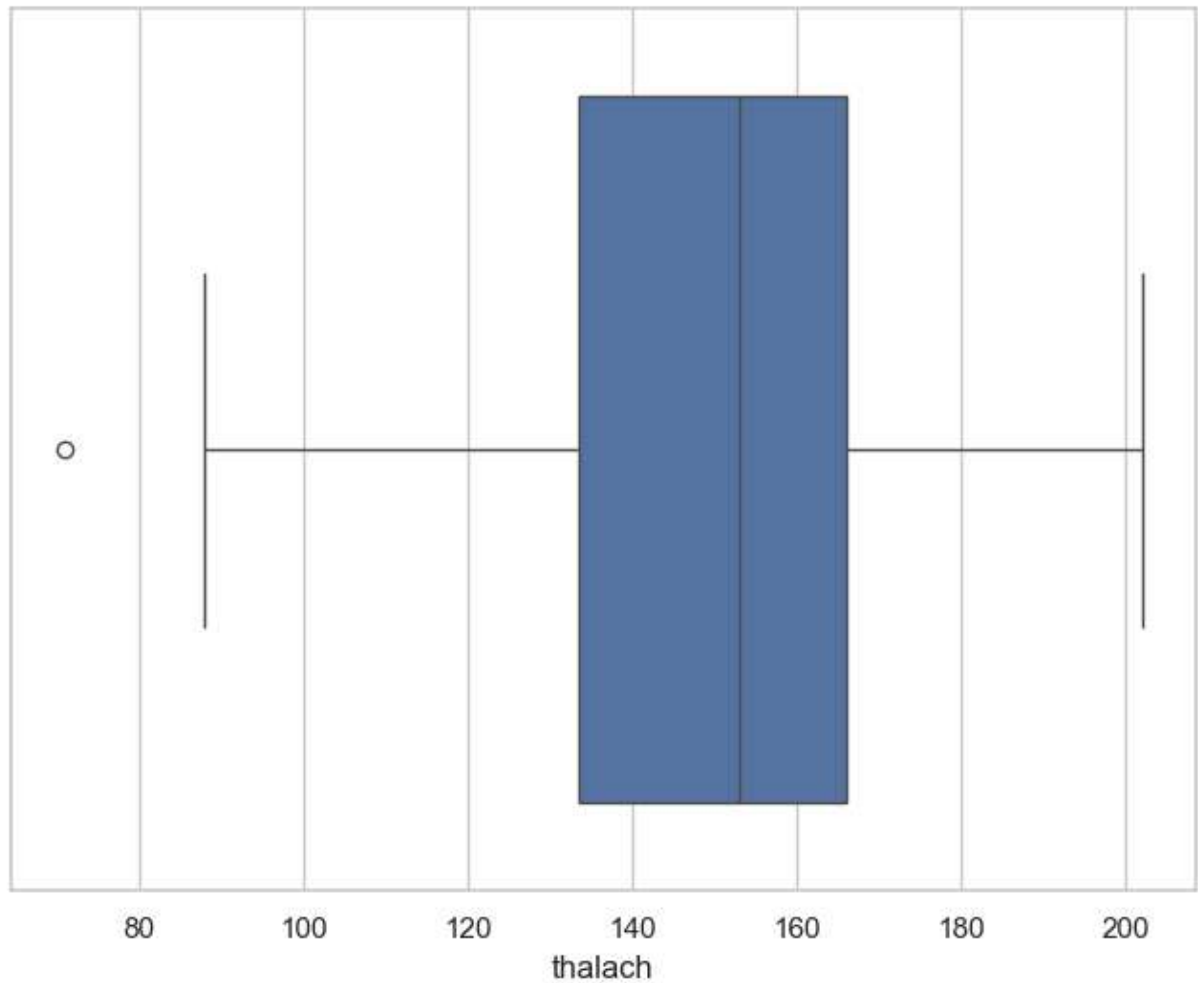
```
In [103...] f, ax = plt.subplots(figsize=(8, 6))
sns.boxplot(x=df["chol"])
plt.show()
```



```
In [104...] df['thalach'].describe()
```

```
Out[104...] count    303.000000  
mean      149.646865  
std        22.905161  
min        71.000000  
25%       133.500000  
50%       153.000000  
75%       166.000000  
max        202.000000  
Name: thalach, dtype: float64
```

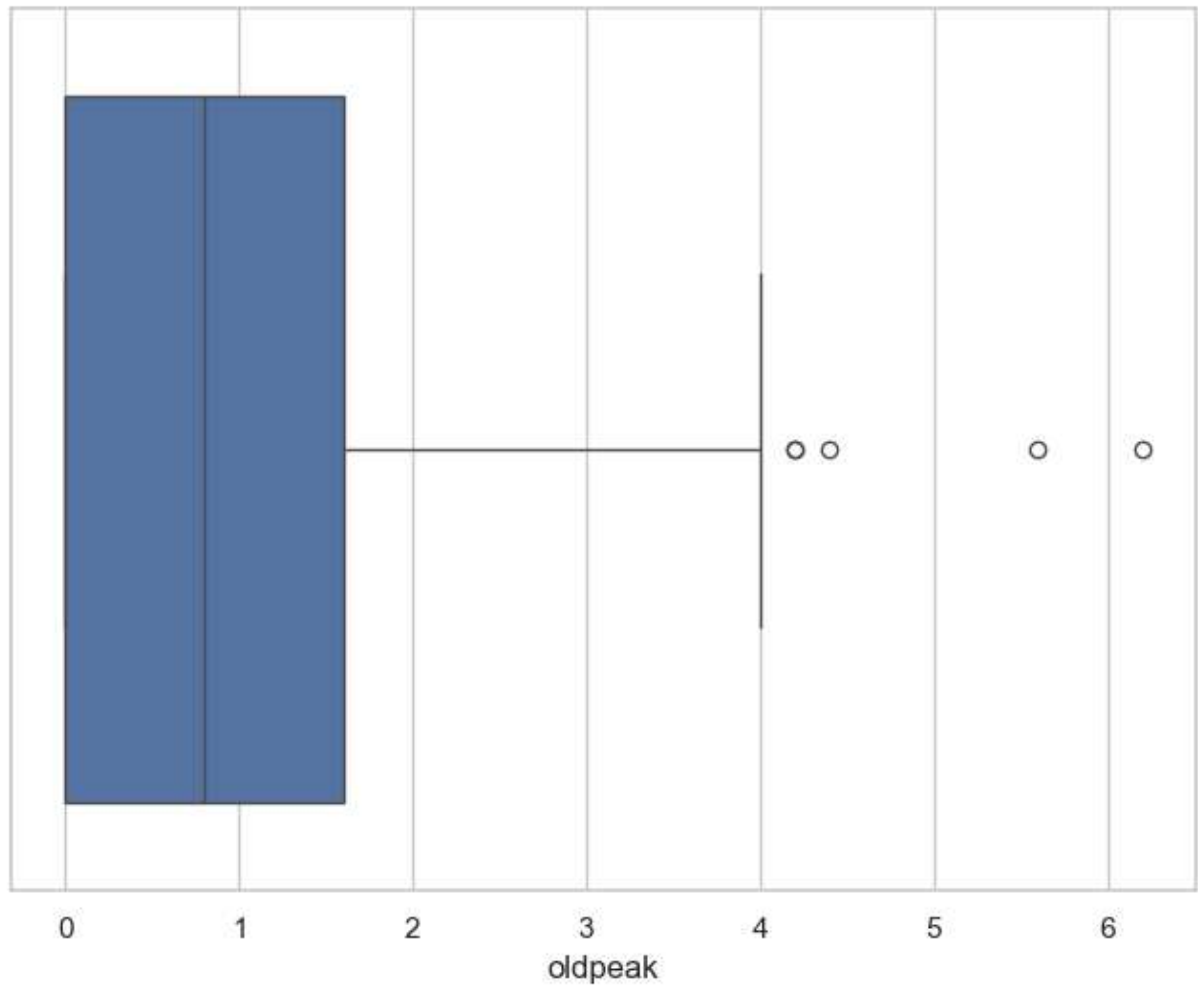
```
In [105...] f, ax = plt.subplots(figsize=(8, 6))  
sns.boxplot(x=df["thalach"])  
plt.show()
```

```
In [106...] df['oldpeak'].describe()
```

```
Out[106...] count    303.000000  
mean       1.039604  
std        1.161075  
min        0.000000  
25%        0.000000  
50%        0.800000  
75%        1.600000  
max        6.200000  
Name: oldpeak, dtype: float64
```

```
In [107...] f, ax = plt.subplots(figsize=(8, 6))  
sns.boxplot(x=df["oldpeak"])  
plt.show()
```



In []:

In []: